UNIVERSIDAD NACIONAL DEL ALTIPLANO

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas

Producto de la I Unidad · Trabajo Encargado N.º 03 y Tarea 6

Simulador de sistemas operativos:

sincronización de procesos en cinco problemas clásicos de concurrencia y planificación de procesos con políticas expulsivas y no expulsivas

Curso:	SIS 215 – Sistemas Operativos
Docente:	Ing. Donia Ruelas
Estudiante:	Emerson Aldair Flores Flores
Código:	240439
Semestre:	2026-II
Fecha:	19 de octubre de 2026

Repositorio del simulador y de los programas:
https://github.com/Enerdev/Sisemas-Operativos

Video explicativo (Google Drive):
https://drive.google.com/drive/folders/1wSo-oogrrC6L8_PlAP1vdnMhg8DpWUAO?usp=sharing

Puno – Perú, 2026

Índice general

Resumen 1

1 Introducción 1

2 Objetivos 2

2.1 Objetivo general . . . 2

2.2 Objetivos específicos . . . 2

3 Marco teórico 3

3.1 Procesos e hilos . . . 3

3.2 Condición de carrera y sección crítica . . . 3

3.3 Mutex, semáforos y monitores . . . 3

3.4 Interbloqueo e inanición . . . 4

3.5 Planificación de CPU . . . 4

3.6 Métricas de la planificación de procesos . . . 5

4 Metodología 5

4.1 Enfoque . . . 5

4.2 Herramientas . . . 5

4.3 Etapas . . . 6

5 Los cinco problemas y sus soluciones 6

5.1 Productor–consumidor . . . 7

5.2 Lectores–escritores . . . 8

5.3 Filósofos comensales . . . 8

5.4 Barbero dormilón . . . 8

5.5 Fumadores . . . 9

6 Diseño del simulador 9

6.1 Arquitectura . . . 9

6.2 Pantalla de inicio y módulos independientes . . . 10

6.3 Elementos comunes de cada caso . . . 10

6.4 Planificador de CPU . . . 11

6.5 Archivos de entrada para casos grandes . . . 13

6.6 Interfaz, temas y diseño adaptable 15

7 Evidencias de implementación: los cinco casos 17

7.1 Caso 1: Productor–consumidor 18

7.2 Caso 2: Lectores–escritores 26

7.3 Caso 3: Filósofos comensales 33

7.4 Caso 4: Barbero dormilón 40

7.5 Caso 5: Fumadores 49

8 Módulo 6: simulador de planificación de procesos (Tarea 6) 56

8.1 Requisitos de la Tarea 6 y cómo se cumplen 56

8.2 Vista general, ingreso de procesos y configuración 57

8.3 Ejecución paso a paso 61

8.4 Resultados por proceso 62

8.5 Las siete políticas con el mismo ejemplo 63

8.6 Comparación de las siete políticas 66

8.7 Situaciones especiales 67

8.8 Código, teoría y tema oscuro 69

9 Resultados: comparación de las políticas de planificación 69

10 Programas en C y Python 72

11 Políticas de uso y privacidad 76

11.1 Políticas de uso 76

11.2 Política de privacidad 76

12 Conclusiones 77

Referencias 78

Anexo A. Repositorio, video y ejecución 80

Anexo B. Claves de los archivos de entrada 81

Anexo C. Tema oscuro de la pantalla de inicio 82

Índice de figuras

1 Pantalla de inicio del simulador 10
2 Configuración de un caso con el planificador de CPU 12
3 Panel del planificador con 2 núcleos y Round Robin 13
4 Línea de tiempo de los filósofos con Round Robin y quantum 1 13
5 Caso cargado desde un archivo con una lista de 2 000 clientes 14
6 Mensajes de error al aplicar un archivo mal escrito 15
7 Caso de los filósofos en tema oscuro 16
8 Vista en un celular: inicio y módulo de planificación de procesos 16
9 Caso 1, Productor–consumidor: vista general del módulo 18
10 Caso 1, Productor–consumidor: panel de configuración 19
11 Caso 1, Productor–consumidor: escena de la solución correcta 20
12 Caso 1, Productor–consumidor: memoria compartida y métricas 20
13 Caso 1, Productor–consumidor: tabla de hilos y bitácora de eventos . . . 21
14 Caso 1, Productor–consumidor: línea de tiempo de los hilos 21
15 Caso 1, Productor–consumidor: falla de la solución incorrecta 22
16 Caso 1, Productor–consumidor: planificador de CPU 22
17 Caso 1, Productor–consumidor: comparación de las 7 políticas 23
18 Caso 1, Productor–consumidor: ejecución con un archivo de entrada grande 24
19 Caso 1, Productor–consumidor: pestañas de código y teoría 25
20 Caso 2, Lectores–escritores: vista general del módulo 26
21 Caso 2, Lectores–escritores: panel de configuración 27
22 Caso 2, Lectores–escritores: escena de la solución correcta 28
23 Caso 2, Lectores–escritores: memoria compartida y métricas 28
24 Caso 2, Lectores–escritores: tabla de hilos y bitácora de eventos 29
25 Caso 2, Lectores–escritores: línea de tiempo de los hilos 29
26 Caso 2, Lectores–escritores: falla de la solución incorrecta 30
27 Caso 2, Lectores–escritores: planificador de CPU 30
28 Caso 2, Lectores–escritores: comparación de las 7 políticas 31
29 Caso 2, Lectores–escritores: ejecución con un archivo de entrada grande . 31
30 Caso 2, Lectores–escritores: pestañas de código y teoría 32
31 Caso 3, Filósofos comensales: vista general del módulo 33
32 Caso 3, Filósofos comensales: panel de configuración 34

33 Caso 3, Filósofos comensales: escena de la solución correcta 35
34 Caso 3, Filósofos comensales: memoria compartida y métricas 35
35 Caso 3, Filósofos comensales: tabla de hilos y bitácora de eventos 36
36 Caso 3, Filósofos comensales: línea de tiempo de los hilos 36
37 Caso 3, Filósofos comensales: falla de la solución incorrecta 37
38 Caso 3, Filósofos comensales: planificador de CPU 37
39 Caso 3, Filósofos comensales: comparación de las 7 políticas 38
40 Caso 3, Filósofos comensales: ejecución con un archivo de entrada grande 38
41 Caso 3, Filósofos comensales: pestañas de código y teoría 39
42 Caso 4, Barbero dormilón: vista general del módulo 40
43 Caso 4, Barbero dormilón: panel de configuración 41
44 Caso 4, Barbero dormilón: escena de la solución correcta 42
45 Caso 4, Barbero dormilón: memoria compartida y métricas 42
46 Caso 4, Barbero dormilón: tabla de hilos y bitácora de eventos 43
47 Caso 4, Barbero dormilón: línea de tiempo de los hilos 43
48 Caso 4, Barbero dormilón: falla de la solución incorrecta 44
49 Caso 4, Barbero dormilón: planificador de CPU 44
50 Caso 4, Barbero dormilón: comparación de las 7 políticas 45
51 Caso 4, Barbero dormilón: ejecución con un archivo de entrada grande . 45
52 Caso 4, Barbero dormilón: pestañas de código y teoría 46
53 Caso 5, Fumadores: vista general del módulo 49
54 Caso 5, Fumadores: panel de configuración 50
55 Caso 5, Fumadores: escena de la solución correcta 51
56 Caso 5, Fumadores: memoria compartida y métricas 51
57 Caso 5, Fumadores: tabla de hilos y bitácora de eventos 52
58 Caso 5, Fumadores: línea de tiempo de los hilos 52
59 Caso 5, Fumadores: falla de la solución incorrecta 53
60 Caso 5, Fumadores: planificador de CPU 53
61 Caso 5, Fumadores: comparación de las 7 políticas 54
62 Caso 5, Fumadores: ejecución con un archivo de entrada grande 54
63 Caso 5, Fumadores: pestañas de código y teoría 55
64 Acceso al módulo 6 desde la pantalla de inicio 57
65 Módulo 6: vista general con Round Robin en $t = 9$ 57

66 Tabla de procesos de entrada . . . 58
67 Panel de configuración del módulo 6 . . . 59
68 Archivo de entrada del módulo 6 . . . 60
69 Estado de la CPU y de las colas en $t = 9$ . . . 61
70 Diagrama de Gantt y orden de ejecución a mitad de la simulación . . . 61
71 Diagrama de estados de cada proceso . . . 61
72 Tabla de resultados durante y al final de la simulación (Round Robin) . . . 62
73 Indicadores globales de la planificación con Round Robin . . . 62
74 Bitácora del planificador . . . 63
75 Diagrama de Gantt de las políticas no expulsivas . . . 64
76 Diagrama de Gantt de las políticas expulsivas . . . 65
77 Comparación de las siete políticas con el ejemplo de clase . . . 66
78 Inanición de P1 con Prioridad expulsiva . . . 67
79 Planificación con dos CPU y HRRN . . . 67
80 Planificación de 1 000 procesos cargados desde un archivo . . . 68
81 Pestañas de código y teoría del módulo 6 . . . 69
82 Módulo 6 en tema oscuro con SRT . . . 69
83 Espera máxima en la cola de listos según la política de planificación . . . 71
84 Salida del programa en C del barbero con atención SRT y 2 núcleos HRRN 75
85 Salida del programa en C de planificación de procesos con Round Robin (quantum 2) . . . 75
86 Políticas de uso y privacidad dentro del simulador . . . 77
87 Pantalla de inicio en tema oscuro . . . 82

Índice de tablas

1 Políticas de planificación implementadas en los cinco casos . . . 4
2 Herramientas usadas en el desarrollo . . . 5
3 Resumen de los cinco problemas clásicos . . . 7
4 Vistas que se muestran para cada caso . . . 17
5 Resultados de los escenarios del caso Productor–Consumidor . . . 25
6 Resultados de los escenarios del caso Lectores–Escritores . . . 32
7 Resultados de los escenarios del caso Filósofos comensales . . . 39
8 Resultados de los escenarios del caso Barbero dormilón . . . 47

9 Comparación de las siete políticas de atención a clientes del barbero . . . 48
10 Resultados de los escenarios del caso Fumadores 55
11 Requisitos de la Tarea 6 y su implementación 56
12 Orden de ejecución del ejemplo de clase con cada política 65
13 Tiempo de espera de cada proceso y promedios del ejemplo de clase . . . 66
14 Comparación de las siete políticas de planificación de CPU en los cinco casos 70
15 Comparación de las siete políticas con 1 000 procesos 72
16 Ejecución de los programas en C y Python con los archivos de ejemplo . 74
17 Contenido del repositorio 80
18 Claves admitidas por cada caso 81

Resumen

Los problemas clásicos de concurrencia resumen las dificultades que aparecen cuando varios hilos comparten recursos: condiciones de carrera, interbloqueo e inanición. Este trabajo presenta un simulador web que reúne en una sola aplicación los cinco problemas propuestos en el curso (productor–consumidor, lectores–escritores, filósofos comensales, barbero dormilón y fumadores), cada uno como un módulo independiente con su propia configuración, escenarios, reloj e historial. Cada módulo muestra una escena ilustrada, el estado de los semáforos y de la memoria compartida, una línea de tiempo por hilo y una bitácora de operaciones `wait` y `signal`. Además, los cinco casos incorporan un planificador de CPU con un número configurable de núcleos y siete políticas: cuatro no expulsivas (FCFS, SJF, Prioridad y HRRN) y tres expulsivas (SRT, Round Robin y Prioridad expulsiva), con su diagrama de Gantt y métricas de cambios de contexto, expulsiones y espera en la cola de listos. El simulador acepta archivos de texto para definir casos con mil o más hilos, y el mismo archivo sirve para los programas equivalentes escritos en C con `pthreads` y en Python con `threading`. Un sexto módulo, desarrollado para la Tarea 6, simula el planificador de procesos: permite ingresar procesos con su llegada, ráfaga y prioridad, aplicar cualquiera de las siete políticas, ver paso a paso el orden de ejecución, la cola de listos y el diagrama de Gantt, y calcular los tiempos de retorno, espera y respuesta de cada proceso; sus resultados coinciden con los de los programas equivalentes en C y Python. Los resultados reproducen los fenómenos esperados: las soluciones correctas no pierden datos ni se bloquean, las versiones sin exclusión mutua presentan condiciones de carrera, las versiones ingenuas de los filósofos y de los fumadores llegan al interbloqueo, y las políticas SJF, SRT y Prioridad aumentan la espera máxima, mientras que HRRN, Round Robin y el envejecimiento la reducen.

Palabras clave: concurrencia, hilos, semáforos, exclusión mutua, interbloqueo, inanición, planificación de procesos, diagrama de Gantt.

1. Introducción

Un sistema operativo ejecuta muchos flujos de ejecución a la vez. Los hilos de un mismo proceso comparten memoria, lo que vuelve rápida la comunicación entre ellos, pero también peligrosa: si dos hilos modifican el mismo dato sin coordinación, el resultado depende del orden en que el planificador los intercale (Ruelas, 2026; Silberschatz et al., 2018). Para estudiar estos riesgos, la literatura usa un conjunto de problemas clásicos que aíslan cada dificultad en una situación concreta: un búfer compartido, un dato que unos leen y otros escriben, una mesa con tenedores escasos, una barbería con sillas limitadas y una mesa de ingredientes (Downey, 2016; Tanenbaum & Bos, 2015).

El Trabajo Encargado N.º 03 pide solucionar estos problemas en cualquier lenguaje de programación y exponerlos en clase. En lugar de cinco programas aislados, este trabajo construye un **simulador único** donde cada problema es un módulo independiente, complementado con programas reales en C y Python que leen los mismos archivos de entrada. Como los problemas de concurrencia se manifiestan en interacción con el planificador del sistema operativo, se incorporó a los cinco casos un planificador de CPU con políticas expulsivas y no expulsivas, lo que conecta este trabajo con el taller de políticas de planificación del curso.

La Tarea 6 pide, además, un simulador de planificación de procesos con interfaz gráfica que permita ingresar procesos, aplicar políticas expulsivas y no expulsivas, mostrar el orden de ejecución, calcular los resultados y dibujar el diagrama de Gantt. Ese simulador se integró como el **módulo 6** de la misma aplicación, de modo que el producto de la unidad cubre tanto la sincronización de procesos como su planificación.

El informe se organiza así: la sección 2 plantea los objetivos; la sección 3 resume los conceptos teóricos; la sección 4 describe la metodología; la sección 5 presenta cada problema y su solución; la sección 6 describe el diseño del simulador; la sección 7 muestra las evidencias de implementación de los cinco casos con la misma estructura; la sección 8 presenta el módulo de planificación de procesos; la sección 9 compara las políticas; la sección 10 describe los programas en C y Python; la sección 11 expone las políticas de uso y privacidad, y la sección 12 presenta las conclusiones.

2. Objetivos

2.1 Objetivo general

Desarrollar un simulador que resuelva y permita analizar los cinco problemas clásicos de concurrencia mediante hilos y primitivas de sincronización, para cualquier número de hilos y bajo distintas políticas de planificación de CPU, e incorporar un simulador de planificación de procesos con políticas expulsivas y no expulsivas.

2.2 Objetivos específicos

1. Modelar cada problema con hilos que comparten recursos y resolverlo con semáforos, mutex o monitores.
2. Implementar cada caso como un módulo independiente, accesible desde una pantalla de inicio.
3. Demostrar de forma reproducible las condiciones de carrera y los interbloqueos que aparecen en las soluciones incorrectas.

4. Incorporar un planificador de CPU con siete políticas, expulsivas y no expulsivas, y medir su efecto en cada caso.
5. Permitir la carga de casos desde archivos de texto con mil o más hilos.
6. Desarrollar un simulador de planificación de procesos (Tarea 6) que permita ingresar procesos, aplicar las siete políticas, mostrar el orden de ejecución y el diagrama de Gantt, y calcular los tiempos de retorno, espera y respuesta.
7. Implementar las mismas soluciones con hilos reales en C (`pthreads`) y Python (`threading`), y el planificador de procesos en C y Python.
8. Definir las políticas de uso y privacidad del simulador conforme a la legislación peruana.

3. Marco teórico

3.1 Procesos e hilos

Un proceso es una instancia de un programa en ejecución, con su propio espacio de direcciones. Un hilo es una unidad de ejecución dentro de un proceso que tiene menos contexto propio: cada hilo tiene su pila, su contador de programa y sus registros, pero comparte con los demás hilos el código, los datos globales y los archivos abiertos (Ruelas, 2026; Arpaci-Dusseau & Arpaci-Dusseau, 2023). El sistema operativo planifica hilos, y puede haber muchos más hilos que núcleos disponibles; por eso solo algunos ejecutan en cada instante y el resto espera en la *cola de listos* (Silberschatz et al., 2018).

3.2 Condición de carrera y sección crítica

Hay condición de carrera cuando dos o más hilos acceden a un dato compartido, al menos uno lo modifica y el resultado depende del orden de ejecución (Ruelas, 2026). La instrucción `contador++`, por ejemplo, se compone de leer, sumar y escribir; si dos hilos la ejecutan intercalada, uno de los incrementos se pierde. La parte del código que accede al recurso compartido es la *sección crítica*, y una solución correcta debe garantizar exclusión mutua, progreso y espera limitada (Maziero, 2019).

3.3 Mutex, semáforos y monitores

Un *semáforo* es un entero con dos operaciones atómicas: `wait` (o `P`), que lo decrementa o bloquea al hilo si vale cero, y `signal` (o `V`), que lo incrementa y despierta a un hilo bloqueado. Dijkstra (1965) los introdujo para coordinar procesos secuenciales que cooperan. Un *mutex* es un semáforo binario que inicia en 1 y protege una sección crítica. Un *monitor* agrupa los datos compartidos con los procedimientos que los manipulan y

garantiza que solo un hilo esté dentro a la vez (Hoare, 1974). Downey (2016) muestra que los cinco problemas de este trabajo pueden resolverse combinando estas tres primitivas.

3.4 Interbloqueo e inanición

Hay interbloqueo cuando un conjunto de hilos queda bloqueado para siempre porque cada uno espera un recurso que retiene otro. Coffman et al. (1971) establecieron que deben cumplirse a la vez cuatro condiciones: exclusión mutua, retención y espera, no expropiación y espera circular; basta romper una para evitarlo. La inanición, en cambio, ocurre cuando un hilo puede avanzar pero nunca le toca, porque otros obtienen siempre la prioridad (Ruelas, 2026). El *envejecimiento* la corrige aumentando la prioridad de un hilo según el tiempo que lleva esperando (Silberschatz et al., 2018).

3.5 Planificación de CPU

Cuando hay más hilos listos que núcleos, el planificador decide a quién despachar. Las políticas se clasifican en *no expulsivas*, en las que un hilo conserva la CPU hasta terminar su ráfaga, y *expulsivas*, en las que el planificador puede quitarle la CPU antes (Silberschatz et al., 2018; Mandl, 2020). La Tabla 1 resume las siete implementadas.

Tabla 1
Políticas de planificación implementadas en los cinco casos

Tipo	**Política**	**Elige al hilo con...**	**Riesgo**
No expulsiva	FCFS	más tiempo en la cola de listos	efecto convoy
No expulsiva	SJF	la ráfaga más corta	inanición de ráfagas largas
No expulsiva	Prioridad	la prioridad más alta (1)	inanición de prioridad baja
No expulsiva	HRRN	mayor razón (espera + ráfaga) / ráfaga	ninguno: la espera sube la razón
Expulsiva	SRT	el menor tiempo restante; expulsa si llega uno menor	muchas expulsiones
Expulsiva	Round Robin	turno circular de un *quantum*	cambios de contexto
Expulsiva	Prioridad expulsiva	la prioridad más alta; expulsa si llega una mayor	inversión de prioridad

Nota. Elaboración propia a partir de Silberschatz et al. (2018) y Arpaci-Dusseau y Arpaci-Dusseau (2023).

3.6 Métricas de la planificación de procesos

Para comparar políticas se usan los tiempos de cada proceso i con llegada L_i, ráfaga R_i, primer despacho I_i y fin F_i (Silberschatz et al., 2018; Wolf et al., 2015):

$$\text{Retorno } TR_i = F_i - L_i, \quad \text{Espera } TE_i = TR_i - R_i, \quad \text{Respuesta } TRes_i = I_i - L_i,$$

y, para el conjunto de n procesos, los promedios $\overline{TR}$, $\overline{TE}$ y $\overline{TRes}$, el *rendimiento* (n entre el tiempo total), el *uso de CPU* (unidades ocupadas entre el tiempo total por el número de CPU) y los *cambios de contexto*. En HRRN la razón de respuesta es $(\text{espera} + R_i)/R_i$; en Round Robin, el *quantum* q es el tiempo máximo que un proceso usa la CPU antes de volver al final de la cola.

4. Metodología

4.1 Enfoque

Se siguió un desarrollo incremental: cada iteración agregó una capacidad al simulador (los cinco casos, la pantalla de inicio, los archivos de entrada, el planificador de CPU y, por último, el módulo de planificación de procesos) y terminó con pruebas antes de pasar a la siguiente. Cada problema se resolvió primero en pseudocódigo con semáforos, luego en el simulador y después con hilos reales en C y Python.

4.2 Herramientas

Tabla 2
Herramientas usadas en el desarrollo

Herramienta	**Uso**
HTML, CSS y JavaScript	Simulador web de un solo archivo, sin servidor ni dependencias.
C con POSIX `pthreads` y GCC	Programas con hilos reales, semáforos y mutex; planificador de procesos en C.
Python 3 con `threading`	Programas equivalentes en Python.
Git y GitHub	Control de versiones y publicación del repositorio.
Playwright y Chromium	Pruebas automáticas de la interfaz y capturas de pantalla del informe.
XeLaTeX	Redacción del informe con formato APA 7.

4.3 Etapas

1. **Análisis:** se identificaron los recursos compartidos, las condiciones de sincronización y los riesgos de cada problema, y los requisitos de la Tarea 6.
2. **Diseño:** se definió una interfaz común para los casos (parámetros, escenarios, estados, paso de simulación, escena, métricas y código) y un motor de planificación único para el módulo 6.
3. **Implementación:** un archivo por caso en el simulador; programas en C y Python que leen el mismo archivo de entrada.
4. **Pruebas:** se ejecutaron todos los escenarios de cada caso con las 7 políticas, con 1, 2 y 4 núcleos y con 3 semillas, comprobando invariantes: ningún ítem perdido o duplicado, la sala nunca supera sus sillas, el dato final es igual al número de escrituras, y nadie queda bloqueado en las soluciones correctas.
5. **Validación cruzada:** los resultados del planificador de procesos se compararon entre el simulador, el programa en C y el programa en Python con los mismos archivos; los tres dieron los mismos tiempos para las 7 políticas. El ejemplo de clase también se calculó a mano (sección 8).
6. **Documentación:** capturas de pantalla automáticas con el mismo conjunto de vistas para cada caso, para que todos los módulos queden documentados por igual.

5. Los cinco problemas y sus soluciones

La Tabla 3 resume qué comparte cada problema, con qué se sincroniza y qué riesgo ilustra.

Tabla 3

Resumen de los cinco problemas clásicos

Caso	Recurso compartido	Primitivas	Riesgo que demuestra
1. Productor–consumidor	búfer circular de N posiciones e índices `in`/`out`	`vacios`, `llenos`, `mutex`	carrera en los índices: ítems perdidos o duplicados
2. Lectores–escritores	un dato que unos leen y otros escriben	`recurso`, `mutex`, `torniquete`	inanición; lecturas sucias
3. Filósofos	N tenedores entre N filósofos	`tenedor[i]`, `sillas`, monitor	interbloqueo por espera circular
4. Barbero dormilón	sala de S sillas y contador `esperando`	`clientes`, `barberos`, `mutex`	carrera en el contador; inanición
5. Fumadores	la mesa con K−1 ingredientes	`agente`, `ingrediente[K]`, `fumador[K]`	interbloqueo de la solución ingenua

Nota. Elaboración propia.

5.1 Productor–consumidor

Uno o más hilos productores generan ítems y los colocan en un búfer de N posiciones; uno o más consumidores los retiran. El productor no puede insertar con el búfer lleno, el consumidor no puede retirar con el búfer vacío, y dos hilos no deben modificar el búfer a la vez (Silberschatz et al., 2018). La solución usa dos semáforos contadores y un mutex (Listado 1). El orden importa: si el productor tomara el mutex antes de `wait(vacios)` y el búfer estuviera lleno, ningún consumidor podría entrar a liberarlo y habría interbloqueo (Downey, 2016).

Listado 1

Productor–consumidor con semáforos

```
semáforo vacios = N, llenos = 0, mutex = 1
hilo Productor:                          hilo Consumidor:
    repetir K veces:                         repetir siempre:
        item = producir()                        wait(llenos)
        wait(vacios)                             wait(mutex)
        wait(mutex)                                  item = buffer[out]
            buffer[in] = item                        out = (out + 1) mod N
            in = (in + 1) mod N                  signal(mutex)
        signal(mutex)                            signal(vacios)
        signal(llenos)                           consumir(item)
```

5.2 Lectores–escritores

Un dato es compartido por lectores, que solo lo consultan, y escritores, que lo modifican. Varios lectores pueden estar dentro a la vez, pero un escritor necesita acceso exclusivo (Courtois et al., 1971). En la solución base, el primer lector toma el semáforo `recurso` en nombre de todos y el último lo libera; esto da *prioridad a los lectores* y puede dejar a los escritores esperando indefinidamente. Courtois et al. (1971) propusieron también una variante con prioridad a los escritores. La versión *justa* agrega un `torniquete` que el escritor cierra mientras espera, de modo que nadie se adelanta y nadie sufre inanición (Downey, 2016). El simulador implementa las tres variantes y una cuarta sin sincronización.

5.3 Filósofos comensales

N filósofos alternan entre pensar y comer alrededor de una mesa con N tenedores; cada uno necesita los dos tenedores que tiene a los lados (Dijkstra, 1965; Tanenbaum & Bos, 2015). La solución ingenua, en la que todos toman primero el izquierdo, cumple las cuatro condiciones de Coffman et al. (1971): si todos lo toman a la vez, se forma una espera circular. El simulador implementa tres soluciones correctas:

- **Asimétrica:** el último filósofo toma primero el tenedor derecho; rompe la espera circular.
- **Camarero:** un semáforo `sillas = N-1` limita cuántos intentan comer; al menos uno consigue ambos tenedores.
- **Monitor de Tanenbaum:** un filósofo come solo si sus vecinos no comen y toma ambos tenedores a la vez; rompe la retención y espera (Tanenbaum & Bos, 2015).

5.4 Barbero dormilón

Una barbería tiene B barberos y S sillas de espera. Sin clientes, los barberos duermen; el cliente que llega despierta a uno o se sienta a esperar, y si la sala está llena se va (Dijkstra, 1965). La solución usa los semáforos `clientes` y `barberos` y un mutex que protege el contador `esperando` (Listado 2). No hay interbloqueo porque el cliente libera el mutex antes de esperar al barbero. Este caso, asignado al equipo, añade siete políticas de *atención a clientes*; con las expulsivas, el cliente sale del sillón con el corte a medias y vuelve a la sala, intercambiando su lugar con quien entra para que la sala no cambie de tamaño.

Listado 2

Barbero dormilón generalizado a B barberos

```
semáforo clientes = 0, barberos = 0, mutex = 1;  entero esperando = 0
hilo Barbero:                          hilo Cliente:
    repetir siempre:                       wait(mutex)
        wait(clientes)                     si esperando < SILLAS:
        wait(mutex)                            esperando = esperando + 1
            esperando = esperando - 1          signal(clientes)
            signal(barberos)                   signal(mutex)
        signal(mutex)                          wait(barberos); recibir_corte()
        cortar_pelo()                      si no: signal(mutex)   // se va
```

5.5 Fumadores

Para armar un cigarro se necesitan tabaco, papel y fósforos; cada fumador tiene una reserva infinita de uno solo. Un agente pone en la mesa dos ingredientes al azar, y el código del agente no puede modificarse (Patil, 1971, como se cita en Parnas, 1975). Si cada fumador espera directamente los ingredientes que le faltan, dos fumadores pueden repartirse la mesa y ninguno completa su cigarro: interbloqueo. Parnas (1975) lo resolvió con hilos *empujadores*, uno por ingrediente, que anotan bajo mutex lo que hay en la mesa y despiertan solo al fumador que puede fumar. El simulador generaliza el problema a K ingredientes (de 3 a 6) e incluye la solución ingenua, la de agente dirigido y la de empujadores.

6. Diseño del simulador

6.1 Arquitectura

El simulador es una aplicación web de un solo archivo (`index.html`) escrita en HTML, CSS y JavaScript, sin dependencias ni servidor. Su código fuente está organizado en un núcleo común y un archivo por caso. Cada caso se define con una misma interfaz: parámetros, escenarios, estados de los hilos, la función que crea la simulación, la función que avanza una unidad de tiempo, la escena ilustrada, las métricas y el código asociado. El núcleo guarda una instancia separada por caso, lo que hace que cada módulo sea **independiente**: lo que se hace en uno no afecta a los demás.

La simulación avanza por *tiempo discreto*. En cada unidad, el caso actualiza a los hilos que trabajan, aplica las operaciones `wait` y `signal` en el orden del algoritmo y registra los eventos; después, el planificador de CPU expulsa y despacha hilos para la unidad siguiente. Las llegadas, duraciones y prioridades se generan con un generador pseudoaleatorio con semilla (*mulberry32*), de modo que cada escenario es reproducible. Cuando se desactiva la exclusión mutua, todos los hilos de una unidad leen el valor compartido del inicio de la unidad y el último en escribir sobrescribe a los demás, que es exactamente una actualización perdida.

6.2 Pantalla de inicio y módulos independientes

La pantalla de inicio (Figura 1) presenta los cinco casos como tarjetas y, debajo, la tarjeta del módulo 6 de planificación de procesos, cada uno con su color e ícono, el estado en que quedó y una tabla comparativa. Para entrar a un caso hay que aceptar las políticas de uso y privacidad. Al entrar, toda la interfaz adopta el color del caso y la cabecera indica «Caso N de 5 · Módulo independiente» (Figura 9).

Figura 1

Pantalla de inicio del simulador



Nota. Las cinco tarjetas de concurrencia muestran el estado de cada caso; debajo están el módulo 6 (Tarea 6), la carga de archivos .txt y la tabla que compara los cinco problemas.

6.3 Elementos comunes de cada caso

Los cinco casos tienen las mismas herramientas de análisis, y la sección 7 las muestra para cada uno en el mismo orden (Tabla 4):

- **Escena ilustrada** con personajes que cambian de color y postura según su estado.
- **Memoria compartida y semáforos**, marcados en rojo cuando su valor deja de coincidir con la realidad.
- **Métricas** del caso y del planificador de CPU.
- **Línea de tiempo** con una fila por hilo y un color por estado.
- **Tabla de hilos** con sus datos privados y **bitácora** de operaciones `wait` y `signal`.
- **Historial** propio, con botones para ejecutar todos los escenarios y comparar las siete políticas.
- Pestañas de **Código** (C, Python y pseudocódigo) y de **Teoría**.

Los controles permiten iniciar, pausar, avanzar una unidad, reiniciar y ejecutar hasta el final, con velocidades de 1 a 1 000 unidades por segundo.

6.4 Planificador de CPU

En la sección «Planificador de CPU» de cada caso se elige el número de núcleos, la política y el *quantum* (Figura 2). Con cero núcleos todos los hilos avanzan a la vez; con n núcleos solo n hilos que tienen trabajo pueden ejecutar en cada unidad. El panel del planificador (Figura 3) muestra los núcleos con el hilo que corre en cada uno, la cola de listos en orden de despacho y el diagrama de Gantt de la CPU. En la línea de tiempo, los tramos amarillos indican que el hilo estaba listo pero esperando CPU (Figura 4). Esperar un semáforo no consume CPU: ese hilo está bloqueado, no listo.

Figura 2

Configuración de un caso con el planificador de CPU



Figura 3

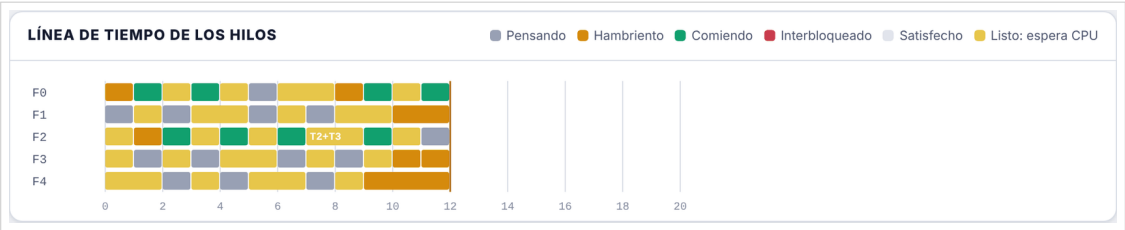
Panel del planificador con 2 núcleos y Round Robin



Nota. Cada núcleo muestra el hilo que ejecuta, su avance y el quantum consumido; la cola de listos está en orden de despacho.

Figura 4

Línea de tiempo de los filósofos con Round Robin y quantum 1



Nota. Los tramos amarillos son las unidades en que un filósofo estuvo listo esperando CPU.

6.5 Archivos de entrada para casos grandes

Los casos pueden definirse en un archivo de texto con una clave por línea (Listado 3). El mismo archivo sirve para el simulador y para los programas en C y Python. El barbero acepta además una lista explícita de clientes (llegada, corte y prioridad) y los fumadores una lista de rondas. El archivo puede cargarse desde la pantalla de inicio, que lo abre en el caso indicado por la línea `caso` (Figura 5), o desde cada caso. Si tiene errores, el simulador indica la línea y el motivo (Figura 6). Los límites del simulador permiten hasta 5 000 filósofos, 5 000 lectores o 50 000 clientes.

Listado 3

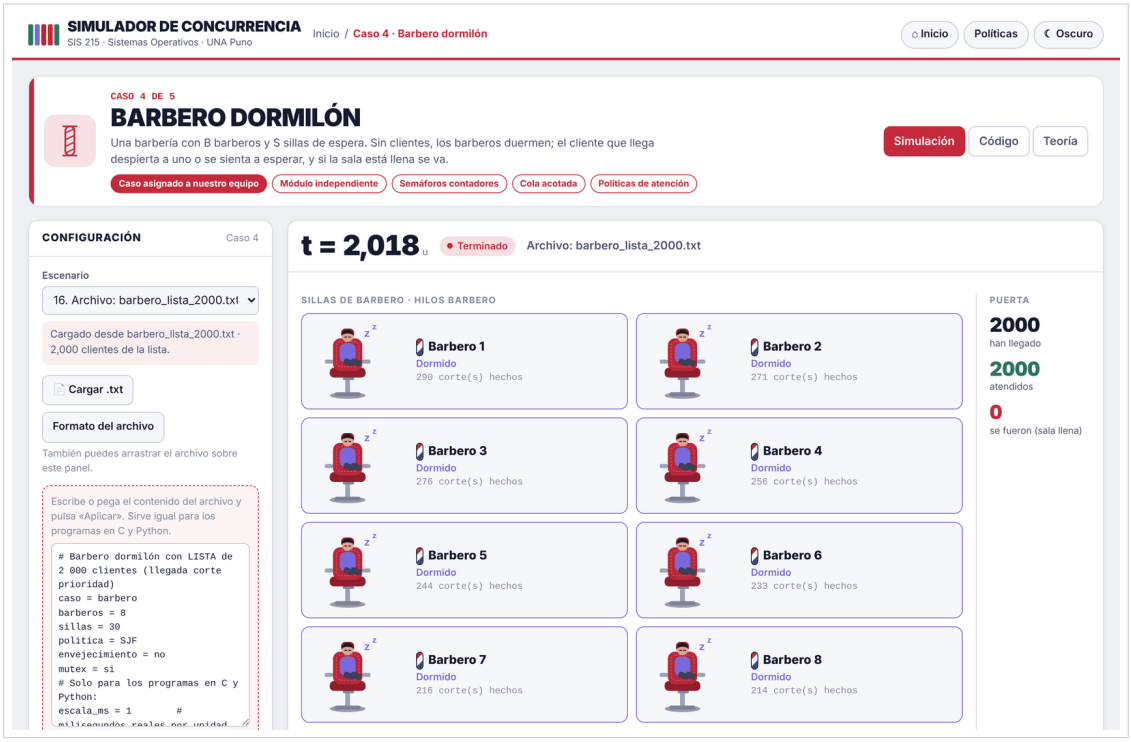
Archivo de entrada para el barbero con 1 000 clientes

```
caso = barbero
barberos = 6
sillas = 25
clientes = 1000
llegada = 0 2          # mínimo y máximo, en unidades de tiempo
corte = 3 12
politica = prioridad   # FIFO | SJF | prioridad | HRRN | SRT | RR |
    prioridad_expulsiva
envejecimiento = si
mutex = si
```

```
10 nucleos = 0              # planificador de CPU: 0 = sin límite
11 planificador = fcfs      # fcfs | sjf | prioridad | hrrn | srt | rr |
       prioridad_expulsiva
12 semilla = 44
13 escala_ms = 1            # solo C/Python: milisegundos reales por unidad
```

Figura 5

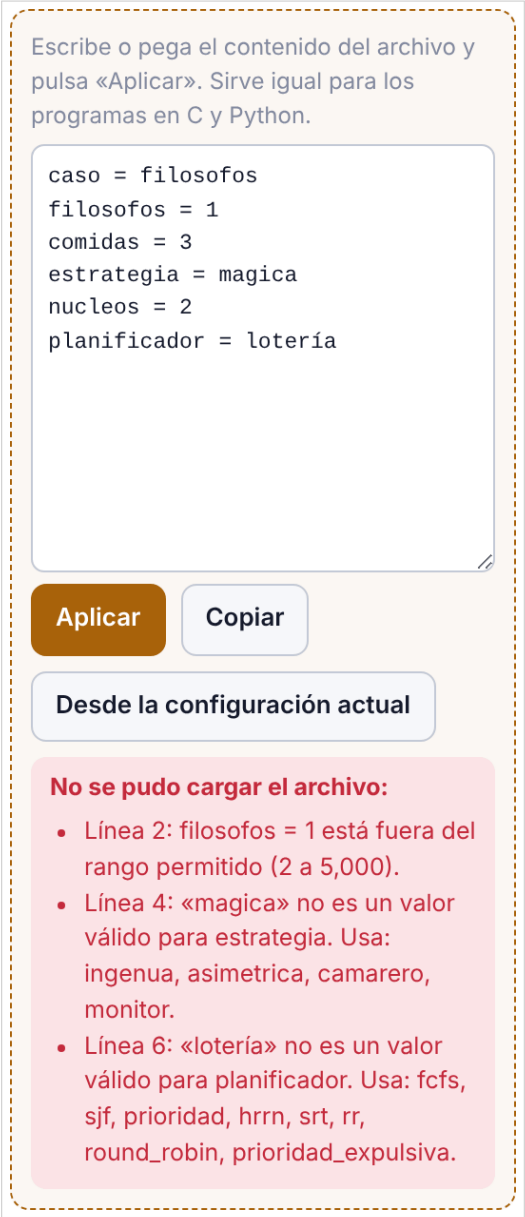
Caso cargado desde un archivo con una lista de 2 000 clientes



Nota. El archivo se convierte en un escenario más del caso; la escena, las métricas y la tabla de clientes se generan a partir de la lista.

Figura 6

Mensajes de error al aplicar un archivo mal escrito



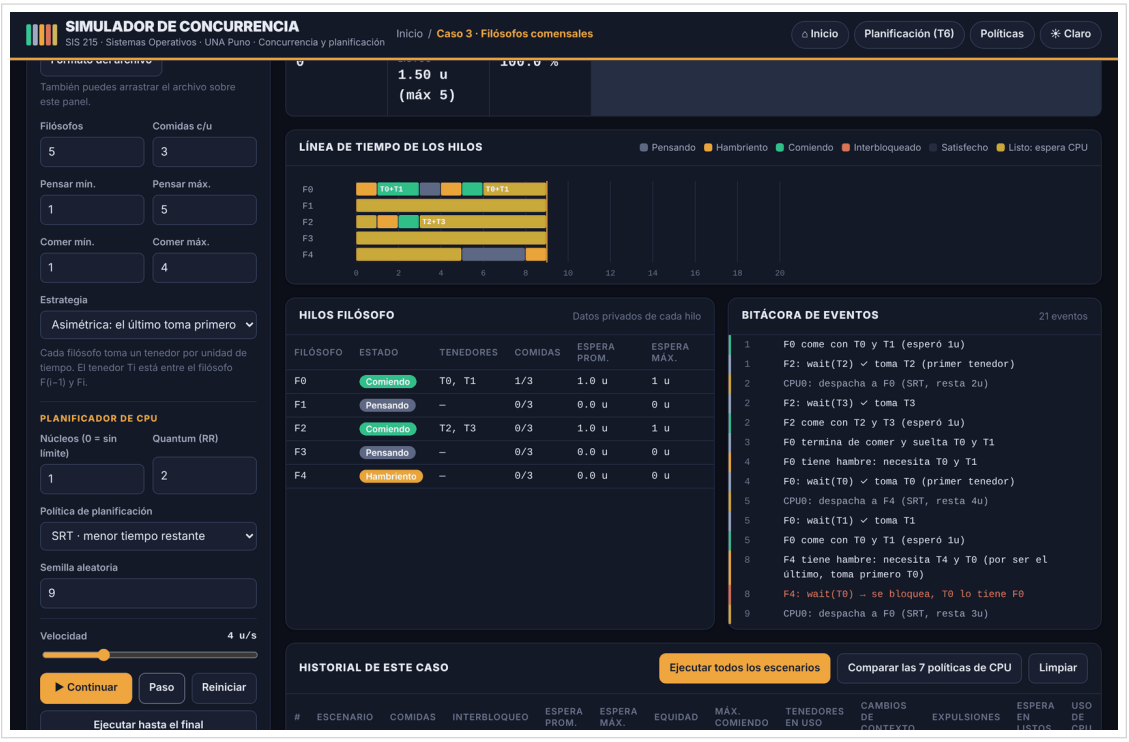
Nota. Cada error indica la línea y el valor esperado.

6.6 Interfaz, temas y diseño adaptable

La interfaz tiene tema claro y oscuro (Figura 7) y se adapta a la pantalla del celular (Figura 8). Cuando hay miles de hilos, la escena dibuja solo los primeros y resume el resto, para que la animación siga siendo fluida.

Figura 7

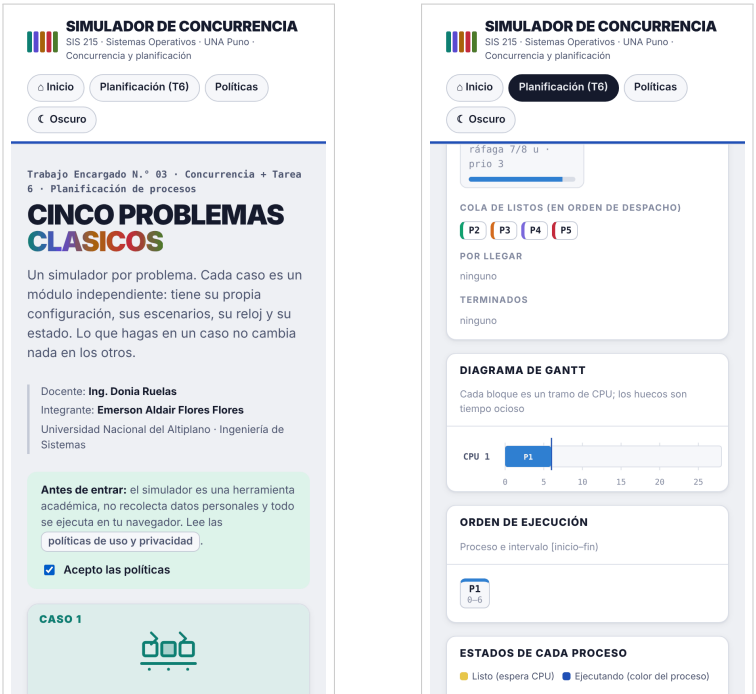
Caso de los filósofos en tema oscuro



Nota. Todos los colores se redefinen para el tema oscuro; el botón superior cambia entre ambos.

Figura 8

Vista en un celular: inicio y módulo de planificación de procesos



Nota. Ancho de 390 píxeles; los paneles se apilan en una sola columna.

7. Evidencias de implementación: los cinco casos

Para que los cinco casos queden documentados por igual, cada uno se presenta con las mismas diez vistas, capturadas con el mismo procedimiento automático, y con la tabla de resultados de todos sus escenarios (Tabla 4). T es el tiempo total en unidades.

Tabla 4
Vistas que se muestran para cada caso

N.º	Vista	Qué demuestra
1	Vista general	Módulo independiente con su color, configuración y escena.
2	Configuración	Escenarios, parámetros, archivo .txt y planificador de CPU.
3	Escena	Funcionamiento de la solución correcta, con ilustraciones.
4	Memoria y métricas	Semáforos, variables compartidas y resultados del escenario.
5	Hilos y bitácora	Datos privados de cada hilo y operaciones `wait/signal`.
6	Línea de tiempo	Estado de cada hilo en cada unidad de tiempo.
7	Falla demostrada	Condición de carrera o interbloqueo de la solución incorrecta.
8	Planificador de CPU	Núcleos, cola de listos y diagrama de Gantt de la CPU.
9	Comparación de políticas	La misma configuración con las 7 políticas.
10	Caso grande y archivo	Ejecución con 1 000 o más hilos cargada desde un .txt.
11	Código y teoría	Programas en C, Python y pseudocódigo, y explicación del caso.

7.1 Caso 1: Productor–consumidor

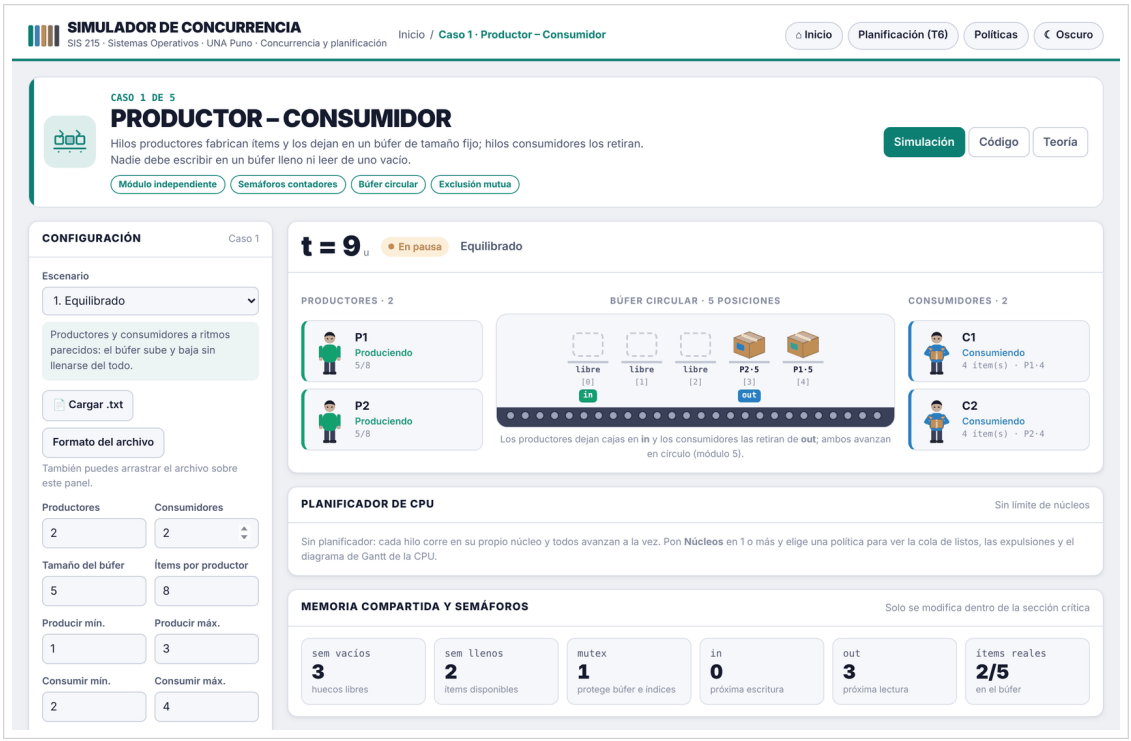
Se usa el escenario «Equilibrado»: 2 productores y 2 consumidores comparten un búfer de 5 posiciones y cada productor fabrica 8 ítems.

7.1.1 Vista general y configuración

La vista se capturó en $t = 9$, con dos cajas en la cinta y los cuatro hilos trabajando. A la izquierda está la configuración del caso (Figura 10).

Figura 9

Caso 1, Productor–consumidor: vista general del módulo



Nota. Cabecera «Caso 1 de 5 · Módulo independiente», configuración a la izquierda y escena a la derecha.

Figura 10

Caso 1, Productor–consumidor: panel de configuración



Nota. Escenario, parámetros, carga de archivo .txt, planificador de CPU y controles de ejecución.

7.1.2 Escena, memoria compartida y métricas

Los productores colocan cajas en la cinta transportadora; los punteros `in` y `out` indican dónde se escribe y de dónde se lee. Los semáforos coinciden con el búfer real: `vacios` = 3, `llenos` = 2 y 2 de 5 posiciones ocupadas.

Figura 11

Caso 1, Productor–consumidor: escena de la solución correcta

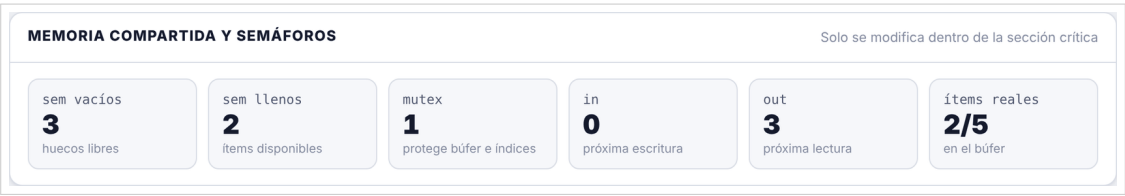


Nota. Los colores de los personajes indican su estado.

Figura 12

Caso 1, Productor–consumidor: memoria compartida y métricas

(a) Memoria compartida y semáforos



(b) Métricas al terminar el escenario



Nota. Los valores en rojo indicarían una inconsistencia; aquí no hay ninguna.

7.1.3 Hilos y bitácora

Cada hilo guarda en privado cuántos ítems produjo o consumió y cuánto tiempo estuvo bloqueado; la bitácora muestra la secuencia `wait(vacios)`, `wait(mutex)`, escritura en el búfer y `signal(llenos)`.

Figura 13

Caso 1, Productor–consumidor: tabla de hilos y bitácora de eventos

(a) Tabla de hilos *(b) Bitácora de eventos*



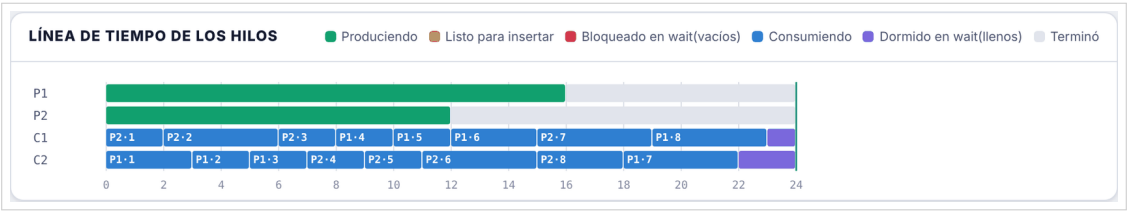
Nota. La tabla guarda los datos privados de cada hilo; la bitácora registra cada operación sobre los semáforos.

7.1.4 Línea de tiempo

Al terminar ($T = 23$ u) se produjeron y consumieron 16 ítems sin errores. En la línea de tiempo cada hilo alterna entre producir o consumir y esperar.

Figura 14

Caso 1, Productor–consumidor: línea de tiempo de los hilos



Nota. Una fila por hilo y un color por estado; la leyenda está en la parte superior.

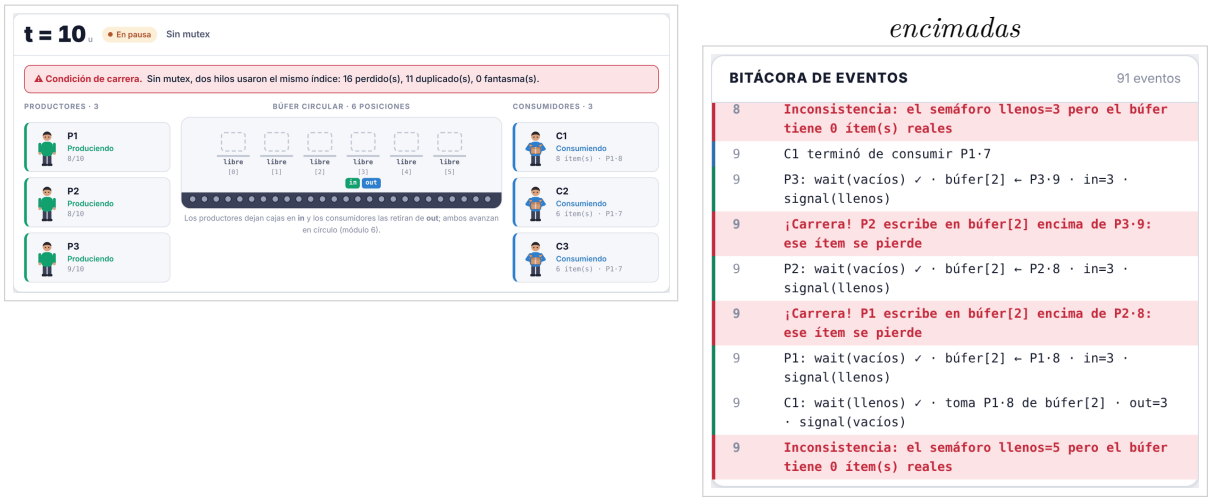
7.1.5 Falla demostrada

Con el escenario «Sin mutex» (3 productores y 3 consumidores), dos hilos usan el mismo índice en la misma unidad: el aviso rojo cuenta los ítems perdidos y duplicados, y la bitácora marca cuando el semáforo `llenos` deja de coincidir con el búfer.

Figura 15

Caso 1, Productor–consumidor: falla de la solución incorrecta

(a) Escena con el aviso de condición de carrera *(b) Bitácora con las escrituras encimadas*



Nota. El simulador detecta la falla automáticamente y la marca en rojo.

7.1.6 Planificador de CPU y comparación de políticas

Con 2 núcleos y Round Robin (quantum 2) solo dos hilos ejecutan a la vez; los demás esperan en la cola de listos y el diagrama de Gantt muestra los turnos de cada núcleo. Las siete políticas consumen los 15 ítems; cambian el tiempo de espera de los consumidores y la ocupación del búfer.

Figura 16

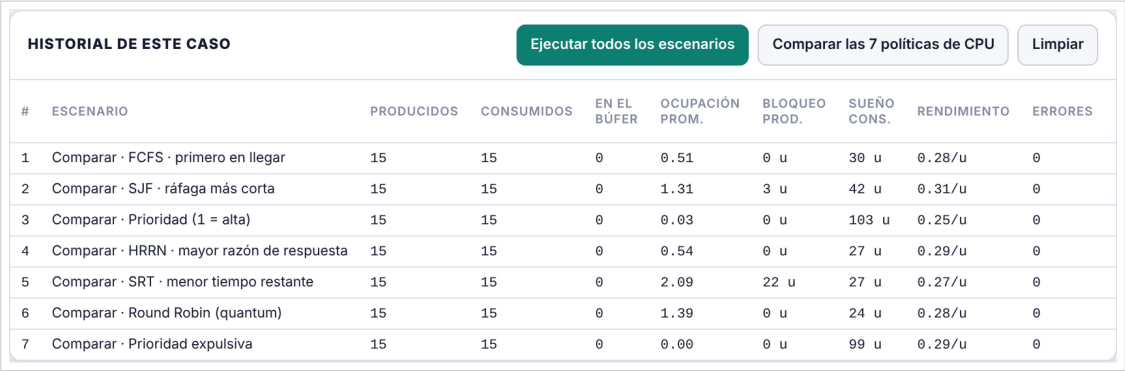
Caso 1, Productor–consumidor: planificador de CPU



Nota. Núcleos con el hilo que ejecuta, cola de listos en orden de despacho y diagrama de Gantt de la CPU.

Figura 17

Caso 1, Productor–consumidor: comparación de las 7 políticas



HISTORIAL DE ESTE CASO

Ejecutar todos los escenarios | Comparar las 7 políticas de CPU | Limpiar

#	ESCENARIO	PRODUCIDOS	CONSUMIDOS	EN EL BÚFER	OCUPACIÓN PROM.	BLOQUEO PROD.	SUEÑO CONS.	RENDIMIENTO	ERRORES
1	Comparar · FCFS · primero en llegar	15	15	0	0.51	0 u	30 u	0.28/u	0
2	Comparar · SJF · ráfaga más corta	15	15	0	1.31	3 u	42 u	0.31/u	0
3	Comparar · Prioridad (1 = alta)	15	15	0	0.03	0 u	103 u	0.25/u	0
4	Comparar · HRRN · mayor razón de respuesta	15	15	0	0.54	0 u	27 u	0.29/u	0
5	Comparar · SRT · menor tiempo restante	15	15	0	2.09	22 u	27 u	0.27/u	0
6	Comparar · Round Robin (quantum)	15	15	0	1.39	0 u	24 u	0.28/u	0
7	Comparar · Prioridad expulsiva	15	15	0	0.00	0 u	99 u	0.29/u	0

Nota. Cada fila es una ejecución completa con la misma configuración y semilla.

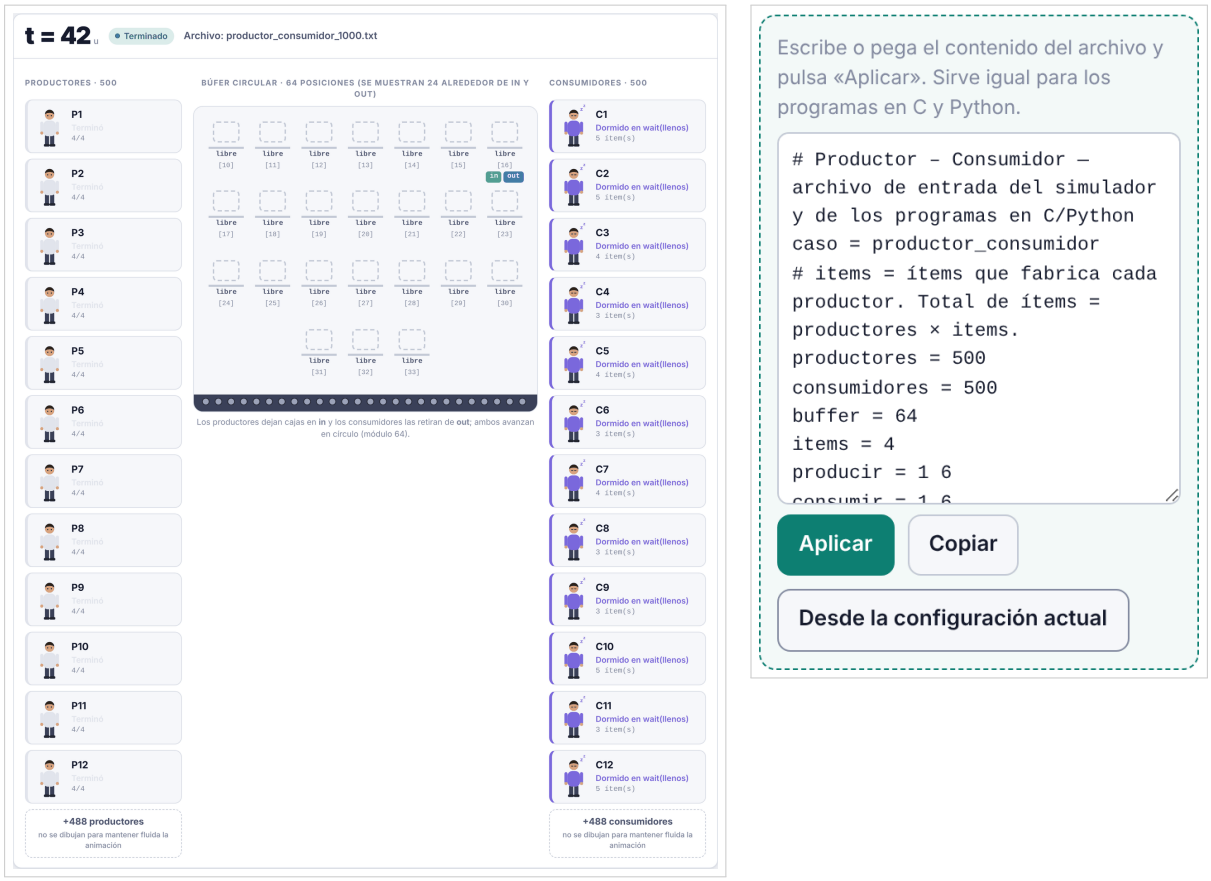
7.1.7 Caso grande cargado desde un archivo

El archivo `productor_consumidor_1000.txt` define 500 productores y 500 consumidores con un búfer de 64 posiciones (2 000 ítems). La simulación terminó en $t = 42$ sin errores.

Figura 18

Caso 1, Productor–consumidor: ejecución con un archivo de entrada grande

(a) Escena al terminar *(b) Formato del archivo*



Nota. Con miles de hilos la escena dibuja los primeros y resume el resto. El mismo archivo sirve para los programas en C y Python.

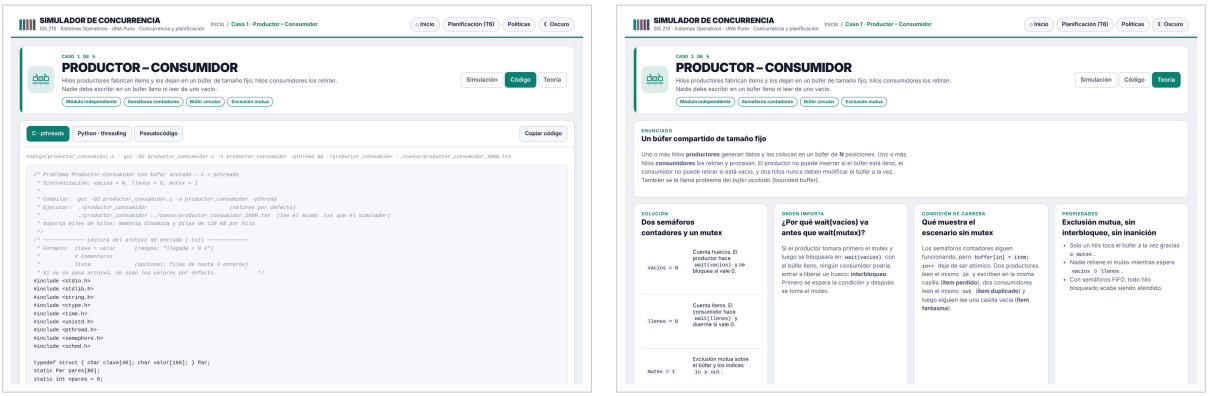
7.1.8 Código y teoría

La pestaña «Código» muestra el programa en C, en Python y el pseudocódigo del caso; la pestaña «Teoría» explica el problema, la solución, qué pasa sin sincronización y el formato del archivo.

Figura 19

Caso 1, Productor–consumidor: pestañas de código y teoría

(a) Código *(b) Teoría*



Nota. El código es el mismo que está en la carpeta `codigo` del repositorio.

7.1.9 Resultados de los escenarios

La Tabla 5 resume los 9 escenarios del caso.

Tabla 5

Resultados de los escenarios del caso Productor–Consumidor

Escenario	T	Producidos	Consumidos	Bloqueo prod.	Sueño cons.	Errores	Cambios ctx.
Equilibrado	23	16	16	0 u	3 u	0	—
Productores rápidos	76	18	18	122 u	1 u	0	—
Consumidores rápidos	44	12	12	0 u	123 u	0	—
Búfer de una posición	13	12	12	2 u	11 u	0	—
Sin mutex	15	30	12	7 u	4 u	**36**	—
CPU · 2 núcleos FCFS	52	15	15	0 u	30 u	0	26
CPU · 2 núcleos Round Robin	53	15	15	0 u	24 u	0	58
CPU · 2 núcleos prioridad expulsiva	51	15	15	0 u	99 u	0	31
Mil hilos	42	2000	2000	5701 u	14463 u	0	—

Nota. T es el tiempo total en unidades. Errores cuenta los ítems perdidos o duplicados; «—» indica que el escenario no usa el planificador de CPU. En rojo, los fallos de sincronización.

7.2 Caso 2: Lectores–escritores

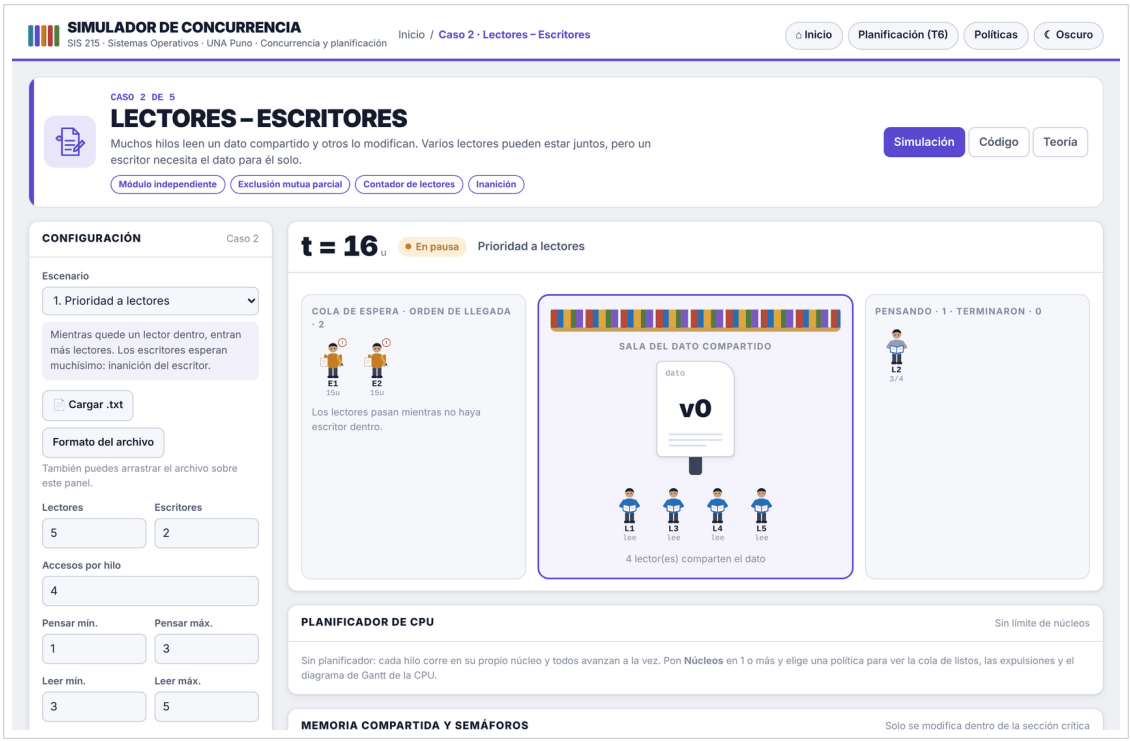
Se usa el escenario «Prioridad a lectores»: 5 lectores y 2 escritores acceden 4 veces cada uno a un dato compartido.

7.2.1 Vista general y configuración

La vista se capturó en $t = 16$: cuatro lectores leen juntos mientras los dos escritores esperan. A la izquierda está la configuración del caso (Figura 21).

Figura 20

Caso 2, Lectores–escritores: vista general del módulo



Nota. Cabecera «Caso 2 de 5 · Módulo independiente», configuración a la izquierda y escena a la derecha.

Figura 21

Caso 2, Lectores–escritores: panel de configuración



Nota. Escenario, parámetros, carga de archivo .txt, planificador de CPU y controles de ejecución.

7.2.2 Escena, memoria compartida y métricas

La sala muestra el documento compartido, los lectores dentro, la cola de espera en orden de llegada y los hilos que piensan. `readcount` = 4 lectores dentro; el semáforo `recurso` está tomado por los lectores y `mutex` = 1 porque nadie modifica el contador.

Figura 22

Caso 2, Lectores–escritores: escena de la solución correcta



Nota. Los colores de los personajes indican su estado.

Figura 23

Caso 2, Lectores–escritores: memoria compartida y métricas

(a) Memoria compartida y semáforos



(b) Métricas al terminar el escenario



Nota. Los valores en rojo indicarían una inconsistencia; aquí no hay ninguna.

7.2.3 Hilos y bitácora

Cada hilo registra sus accesos y esperas; la bitácora muestra al primer lector tomando `recurso` en nombre de todos y al último liberándolo.

Figura 24

Caso 2, Lectores–escritores: tabla de hilos y bitácora de eventos

(a) Tabla de hilos *(b) Bitácora de eventos*



HILOS LECTORES Y ESCRITORES — Datos privados de cada hilo

HILO	ROL	ESTADO	ACCESOS	ESPERA PROM.	ESPERA MÁX.
L1	Lector	Leyendo	3/4	0.0 u	0 u
L2	Lector	Pensando	3/4	0.0 u	0 u
L3	Lector	Leyendo	3/4	0.0 u	0 u
L4	Lector	Leyendo	3/4	0.0 u	0 u
L5	Lector	Leyendo	2/4	0.0 u	0 u
E1	Escritor	Esperando el dato	0/4	0.0 u	15 u
E2	Escritor	Esperando el dato	0/4	0.0 u	15 u

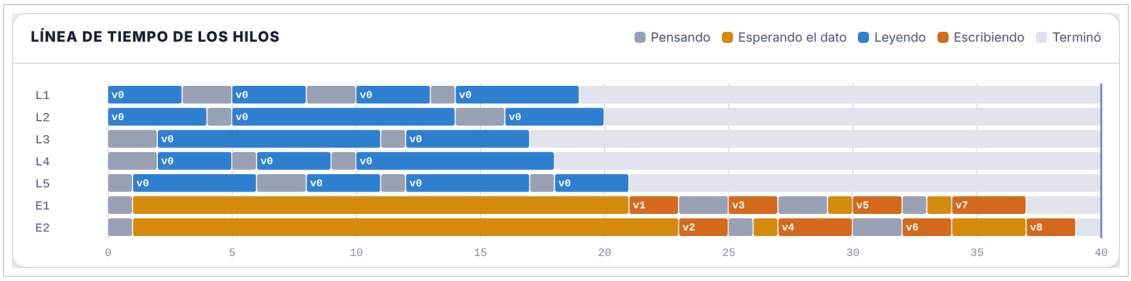
Nota. La tabla guarda los datos privados de cada hilo; la bitácora registra cada operación sobre los semáforos.

7.2.4 Línea de tiempo

Al terminar ($T = 39$ u) hubo 20 lecturas y 8 escrituras; la línea de tiempo muestra las esperas largas de los escritores, propias de esta política.

Figura 25

Caso 2, Lectores–escritores: línea de tiempo de los hilos



Nota. Una fila por hilo y un color por estado; la leyenda está en la parte superior.

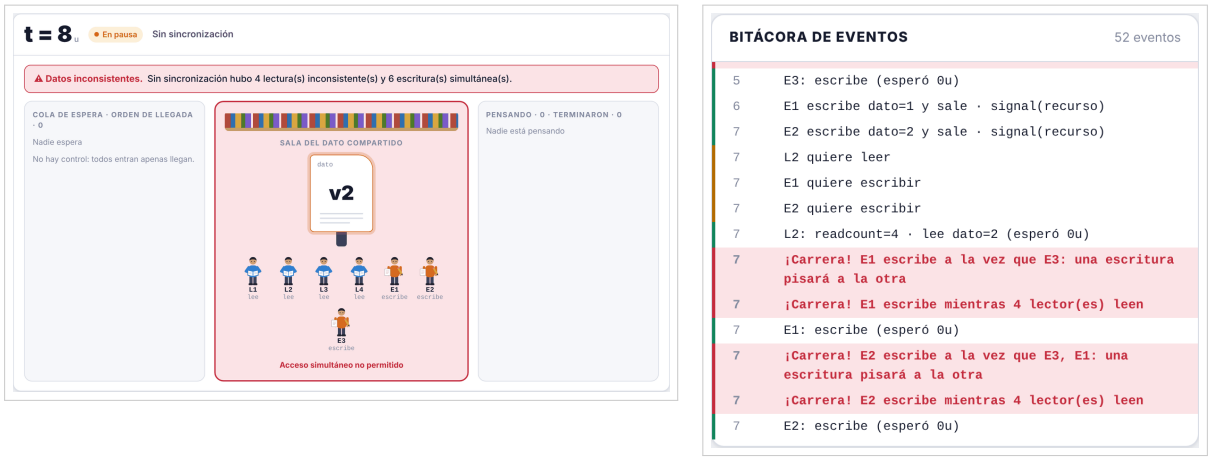
7.2.5 Falla demostrada

Con el escenario «Sin sincronización» los escritores entran mientras otros escriben o leen: el aviso cuenta 4 lecturas inconsistentes y 6 escrituras simultáneas, y la bitácora marca cada carrera.

Figura 26

Caso 2, Lectores–escritores: falla de la solución incorrecta

(a) Escena con acceso simultáneo no permitido *(b) Bitácora con las carreras detectadas*



Nota. El simulador detecta la falla automáticamente y la marca en rojo.

7.2.6 Planificador de CPU y comparación de políticas

Con un solo núcleo y Round Robin los hilos se turnan la CPU para pensar, leer y escribir; el diagrama de Gantt muestra el reparto. Las siete políticas completan todas las lecturas y escrituras; varían el tiempo total y la espera de lectores y escritores.

Figura 27

Caso 2, Lectores–escritores: planificador de CPU



Nota. Núcleos con el hilo que ejecuta, cola de listos en orden de despacho y diagrama de Gantt de la CPU.

Figura 28

Caso 2, Lectores–escritores: comparación de las 7 políticas



HISTORIAL DE ESTE CASO

Ejecutar todos los escenarios | Comparar las 7 políticas de CPU | Limpiar

#	ESCENARIO	LECTURAS	ESCRITURAS	ESPERA PROM. L	ESPERA PROM. E	ESPERA MÁX. E	MÁX. LECTORES JUNTOS	LECT. INCONSISTENTES	ESCRIT. PERDIDAS
1	Comparar · FCFS · primero en llegar	12	6	6.50 u	15.00 u	17 u	4	0	0
2	Comparar · SJF · ráfaga más corta	12	6	9.33 u	15.00 u	18 u	4	0	0
3	Comparar · Prioridad (1 = alta)	12	6	2.00 u	4.17 u	7 u	2	0	0
4	Comparar · HRRN · mayor razón de respuesta	12	6	8.92 u	14.67 u	17 u	4	0	0
5	Comparar · SRT · menor tiempo restante	12	6	9.33 u	15.00 u	18 u	4	0	0
6	Comparar · Round Robin (quantum)	12	6	5.67 u	12.17 u	15 u	4	0	0
7	Comparar · Prioridad expulsiva	12	6	2.00 u	4.17 u	7 u	2	0	0

Nota. Cada fila es una ejecución completa con la misma configuración y semilla.

7.2.7 Caso grande cargado desde un archivo

El archivo `lectores_escritores_1000.txt` define 800 lectores y 200 escritores con 3 accesos cada uno y la política justa. Terminó en $t = 2\,215$ con el dato en v600, es decir, las 600 escrituras sin pérdidas.

Figura 29

Caso 2, Lectores–escritores: ejecución con un archivo de entrada grande

(a) Escena al terminar *(b) Formato del archivo*



t = 2,215 · Terminado · Archivo: lectores_escritores_1000.txt

COLA DE ESPERA · ORDEN DE LLEGADA · 0

Nadie espera

El primero de la cola decide: si es escritor, cierra el torniquete.

SALA DEL DATO COMPARTIDO

dato

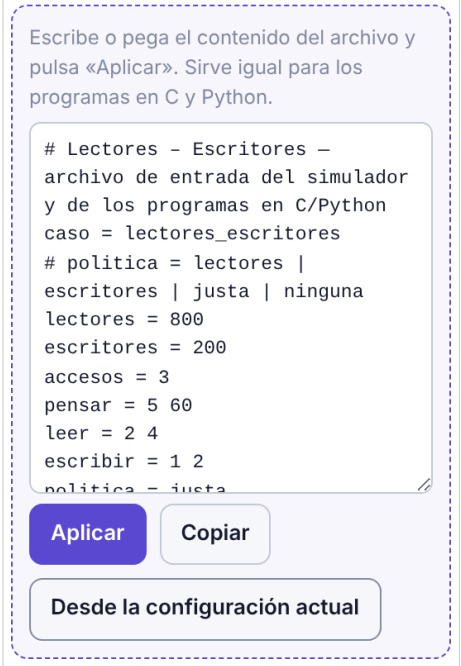
v600

Nadie dentro

Recurso libre

PENSANDO · 0 · TERMINARON · 1000

Nadie está pensando



Escribe o pega el contenido del archivo y pulsa «Aplicar». Sirve igual para los programas en C y Python.

```
# Lectores - Escritores —
archivo de entrada del simulador
y de los programas en C/Python
caso = lectores_escritores
# politica = lectores |
escritores | justa | ninguna
lectores = 800
escritores = 200
accesos = 3
pensar = 5 60
leer = 2 4
escribir = 1 2
politica = justa
```

Aplicar | Copiar

Desde la configuración actual

Nota. Con miles de hilos la escena dibuja los primeros y resume el resto. El mismo archivo sirve para los programas en C y Python.

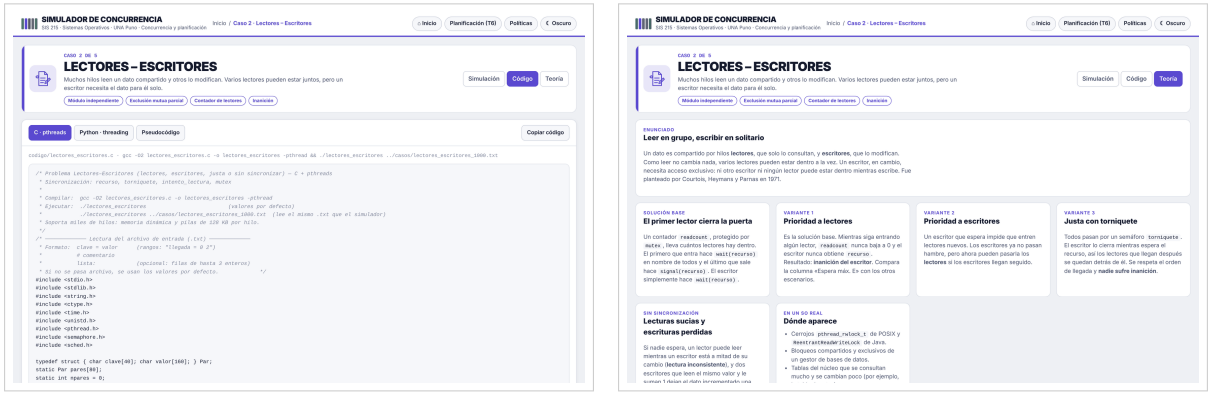
7.2.8 Código y teoría

La pestaña «Código» muestra el programa en C, en Python y el pseudocódigo del caso; la pestaña «Teoría» explica el problema, la solución, qué pasa sin sincronización y el formato del archivo.

Figura 30

Caso 2, Lectores–escritores: pestañas de código y teoría

(a) Código *(b) Teoría*



Nota. El código es el mismo que está en la carpeta `codigo` del repositorio.

7.2.9 Resultados de los escenarios

La Tabla 6 resume los 9 escenarios del caso.

Tabla 6

Resultados de los escenarios del caso Lectores–Escritores

Escenario	T	Lecturas	Escrituras	Espera prom. L	Espera prom. E	Espera máx. E	Lect. sucias	Escr. perdidas
Prioridad a lectores	39	20	8	0.00 u	6.00 u	22 u	0	0
Prioridad a escritores	46	20	8	5.30 u	2.75 u	5 u	0	0
Justa (torniquete)	46	20	8	5.05 u	7.00 u	8 u	0	0
Muchos escritores	66	12	16	9.00 u	10.88 u	15 u	0	0
Sin sincronización	21	16	12	0.00 u	0.00 u	0 u	**15**	**7**
CPU · 2 núcleos prioridad expulsiva	42	12	6	3.67 u	7.33 u	11 u	0	0
CPU · 1 núcleo Round Robin	70	12	6	5.67 u	12.17 u	15 u	0	0
CPU · 2 núcleos HRRN	46	12	6	5.50 u	8.50 u	10 u	0	0
Mil hilos	2215	2400	600	546.39 u	557.74 u	834 u	0	0

Nota. T es el tiempo total en unidades. Las esperas son el tiempo entre pedir el acceso y obtenerlo. En rojo, los fallos de sincronización.

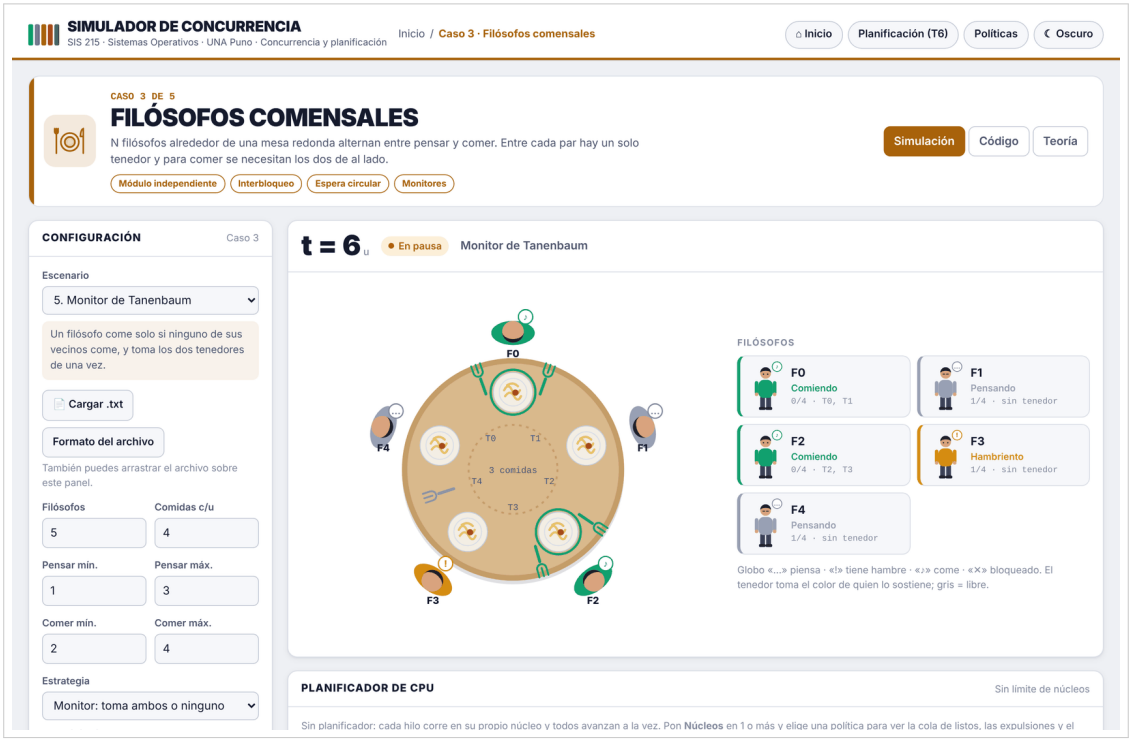
7.3 Caso 3: Filósofos comensales

Se usa el escenario «Monitor de Tanenbaum»: 5 filósofos comen 4 veces cada uno.

7.3.1 Vista general y configuración

La vista se capturó en $t = 6$: F0 y F2 comen y F3 tiene hambre. A la izquierda está la configuración del caso (Figura 32).

Figura 31
Caso 3, Filósofos comensales: vista general del módulo



Nota. Cabecera «Caso 3 de 5 · Módulo independiente», configuración a la izquierda y escena a la derecha.

Figura 32

Caso 3, Filósofos comensales: panel de configuración



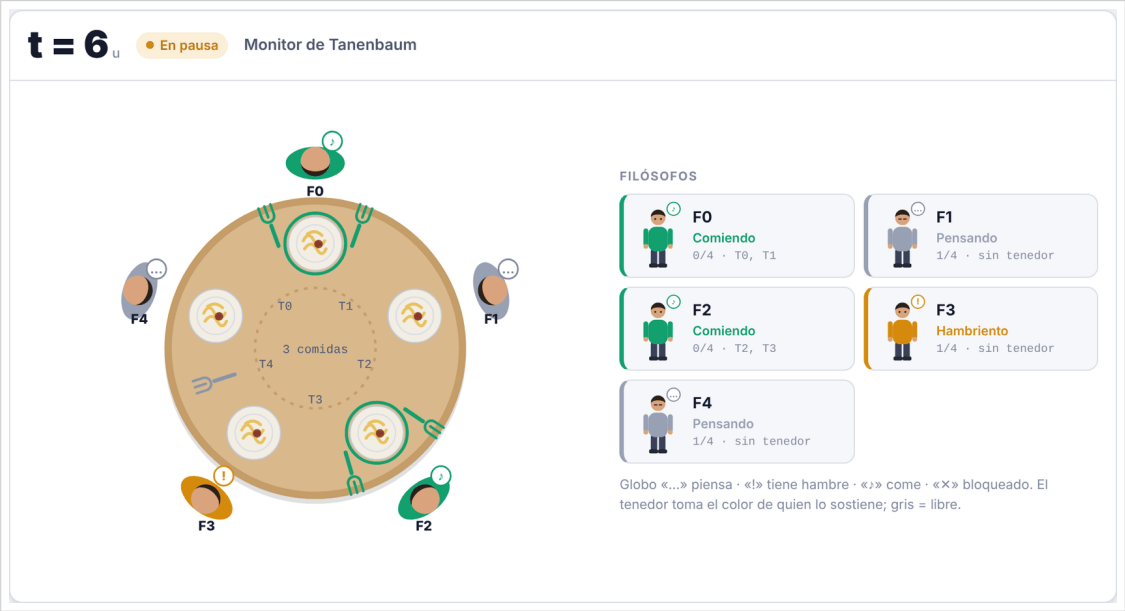
Nota. Escenario, parámetros, carga de archivo .txt, planificador de CPU y controles de ejecución.

7.3.2 Escena, memoria compartida y métricas

La mesa muestra los platos, los tenedores (que se desplazan hacia quien los toma) y un globo con el estado de cada filósofo. Hay 1 de 5 tenedores libre, 2 filósofos comiendo (el máximo posible con 5) y 1 hambriento esperando; el monitor deja entrar a un hilo a la vez.

Figura 33

Caso 3, Filósofos comensales: escena de la solución correcta



Nota. Los colores de los personajes indican su estado.

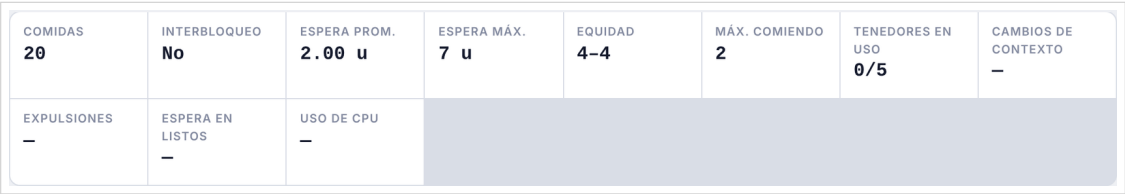
Figura 34

Caso 3, Filósofos comensales: memoria compartida y métricas

(a) Memoria compartida y semáforos



(b) Métricas al terminar el escenario



Nota. Los valores en rojo indicarían una inconsistencia; aquí no hay ninguna.

7.3.3 Hilos y bitácora

Cada filósofo guarda cuántas veces comió y qué tenedores tiene; la bitácora muestra cuándo prueba sus vecinos y cuándo toma ambos tenedores a la vez.

Figura 35

Caso 3, Filósofos comensales: tabla de hilos y bitácora de eventos

(a) Tabla de hilos *(b) Bitácora de eventos*



HILOS FILÓSOFO — Datos privados de cada hilo

FILÓSOFO	ESTADO	TENEDORES	COMIDAS	ESPERA PROM.	ESPERA MÁX.
F0	Comiendo	T0, T1	0/4	3.0 u	3 u
F1	Pensando	—	1/4	0.0 u	0 u
F2	Comiendo	T2, T3	0/4	3.0 u	3 u
F3	Hambriento	—	1/4	0.0 u	0 u
F4	Pensando	—	1/4	1.0 u	1 u

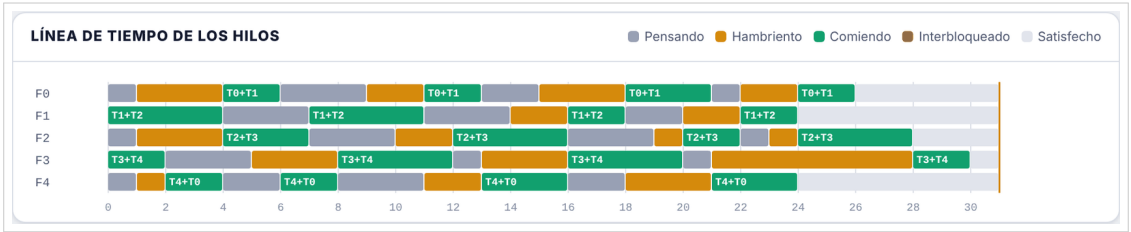
Nota. La tabla guarda los datos privados de cada hilo; la bitácora registra cada operación sobre los semáforos.

7.3.4 Línea de tiempo

Al terminar ($T = 30$ u) hubo 20 comidas, sin interbloqueo y con un reparto equitativo de 4 comidas por filósofo.

Figura 36

Caso 3, Filósofos comensales: línea de tiempo de los hilos



Nota. Una fila por hilo y un color por estado; la leyenda está en la parte superior.

7.3.5 Falla demostrada

Con la estrategia ingenua y tiempos iguales, en $t = 1$ los cinco toman su tenedor izquierdo: el simulador detecta la espera circular, la dibuja en rojo y la describe en la bitácora.

Figura 37

Caso 3, Filósofos comensales: falla de la solución incorrecta

(a) Mesa con el interbloqueo *(b) Bitácora con la espera circular*



Nota. El simulador detecta la falla automáticamente y la marca en rojo.

7.3.6 Planificador de CPU y comparación de políticas

Con 2 núcleos y Round Robin de quantum 1, como máximo dos filósofos usan la CPU por unidad; los que comen sin CPU retienen sus tenedores más tiempo. Las siete políticas completan las comidas sin interbloqueo; cambian el tiempo total y la espera en la cola de listos.

Figura 38

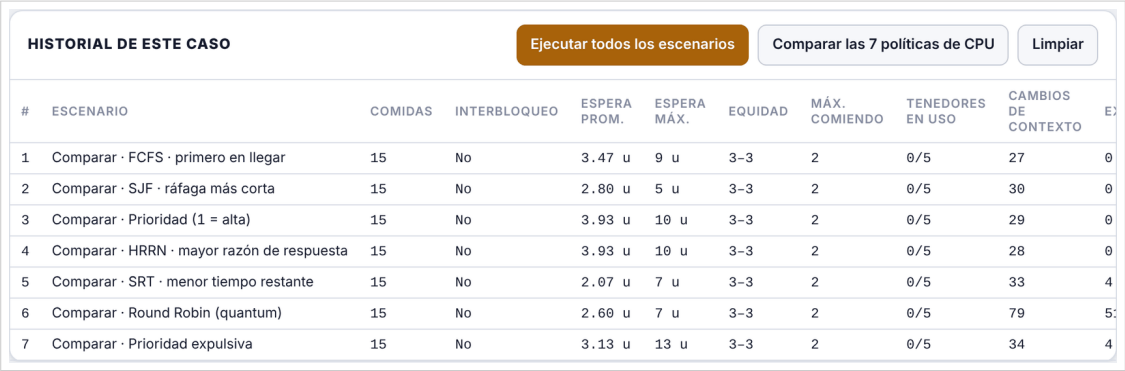
Caso 3, Filósofos comensales: planificador de CPU



Nota. Núcleos con el hilo que ejecuta, cola de listos en orden de despacho y diagrama de Gantt de la CPU.

Figura 39

Caso 3, Filósofos comensales: comparación de las 7 políticas



#	ESCENARIO	COMIDAS	INTERBLOQUEO	ESPERA PROM.	ESPERA MÁX.	EQUIDAD	MÁX. COMIENDO	TENEDORES EN USO	CAMBIOS DE CONTEXTO	E
1	Comparar · FCFS · primero en llegar	15	No	3.47 u	9 u	3–3	2	0/5	27	0
2	Comparar · SJF · ráfaga más corta	15	No	2.80 u	5 u	3–3	2	0/5	30	0
3	Comparar · Prioridad (1 = alta)	15	No	3.93 u	10 u	3–3	2	0/5	29	0
4	Comparar · HRRN · mayor razón de respuesta	15	No	3.93 u	10 u	3–3	2	0/5	28	0
5	Comparar · SRT · menor tiempo restante	15	No	2.07 u	7 u	3–3	2	0/5	33	4
6	Comparar · Round Robin (quantum)	15	No	2.60 u	7 u	3–3	2	0/5	79	5
7	Comparar · Prioridad expulsiva	15	No	3.13 u	13 u	3–3	2	0/5	34	4

Nota. Cada fila es una ejecución completa con la misma configuración y semilla.

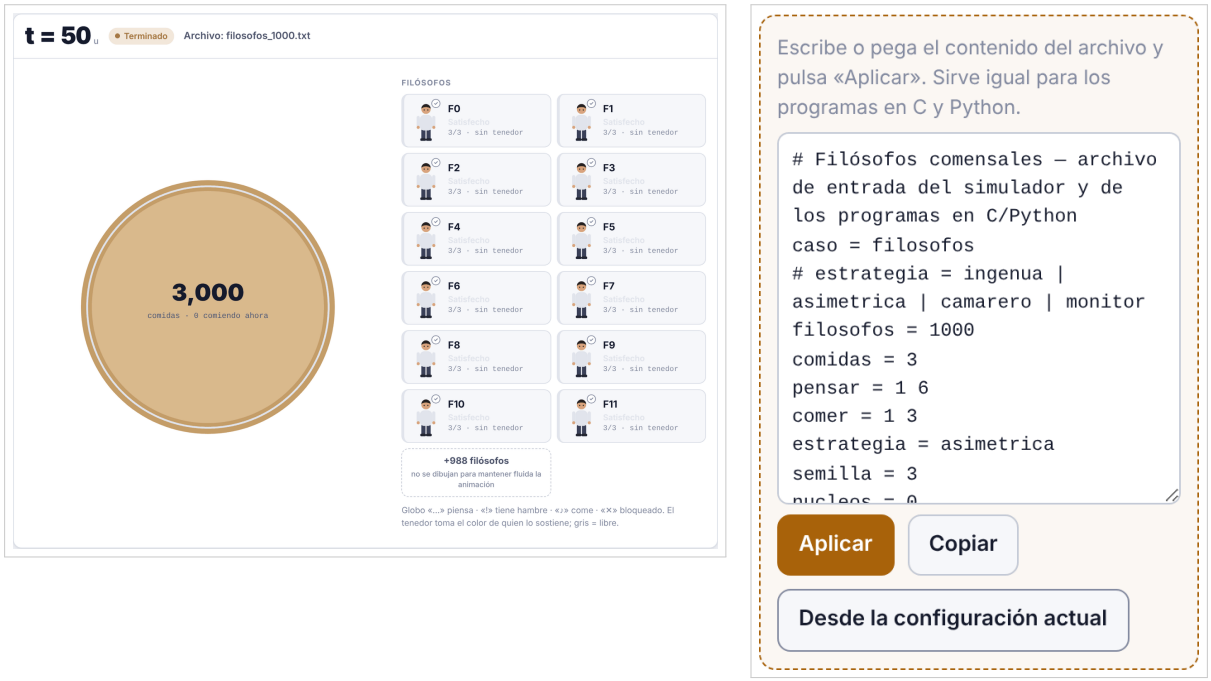
7.3.7 Caso grande cargado desde un archivo

El archivo `filosofos_1000.txt` define 1 000 filósofos con 3 comidas y la estrategia asimétrica. Terminó en $t = 50$ con 3 000 comidas y sin interbloqueo.

Figura 40

Caso 3, Filósofos comensales: ejecución con un archivo de entrada grande

(a) Escena al terminar *(b) Formato del archivo*



Nota. Con miles de hilos la escena dibuja los primeros y resume el resto. El mismo archivo sirve para los programas en C y Python.

7.3.8 Código y teoría

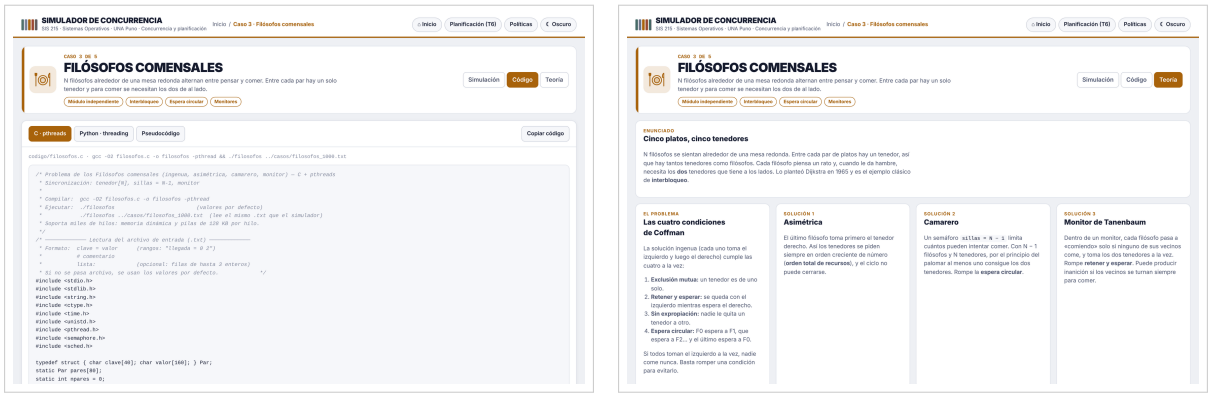
La pestaña «Código» muestra el programa en C, en Python y el pseudocódigo del caso; la pestaña «Teoría» explica el problema, la solución, qué pasa sin sincronización y

el formato del archivo.

Figura 41

Caso 3, Filósofos comensales: pestañas de código y teoría

(a) Código *(b) Teoría*



Nota. El código es el mismo que está en la carpeta `codigo` del repositorio.

7.3.9 Resultados de los escenarios

La Tabla 7 resume los 10 escenarios del caso.

Tabla 7

Resultados de los escenarios del caso Filósofos comensales

Escenario	T	Comidas	Interbloqueo	Espera prom.	Espera máx.	Máx. comiendo	Cambios ctx.
Ingenua: interbloqueo	1	0	Sí (t=1)	0.00 u	0 u	0	—
Ingenua con tiempos aleatorios	43	24	Sí (t=43)	4.88 u	10 u	2	—
Asimétrica	28	20	No	2.35 u	7 u	2	—
Camarero (N−1 sillas)	43	20	No	6.00 u	10 u	1	—
Monitor de Tanenbaum	30	20	No	2.00 u	7 u	2	—
Mesa de diez	45	50	No	2.78 u	11 u	4	—
CPU · 2 núcleos SJF	49	15	No	2.80 u	5 u	2	30
CPU · 2 núcleos Round Robin q=1	45	15	No	2.60 u	7 u	2	79
CPU · 1 núcleo SRT	86	15	No	1.40 u	4 u	2	33
Mil filósofos	50	3000	No	3.58 u	24 u	246	—

Nota. T es el tiempo total en unidades. «Máx. comiendo» es el mayor número de filósofos comiendo a la vez. En rojo, los fallos de sincronización.

7.4 Caso 4: Barbero dormilón

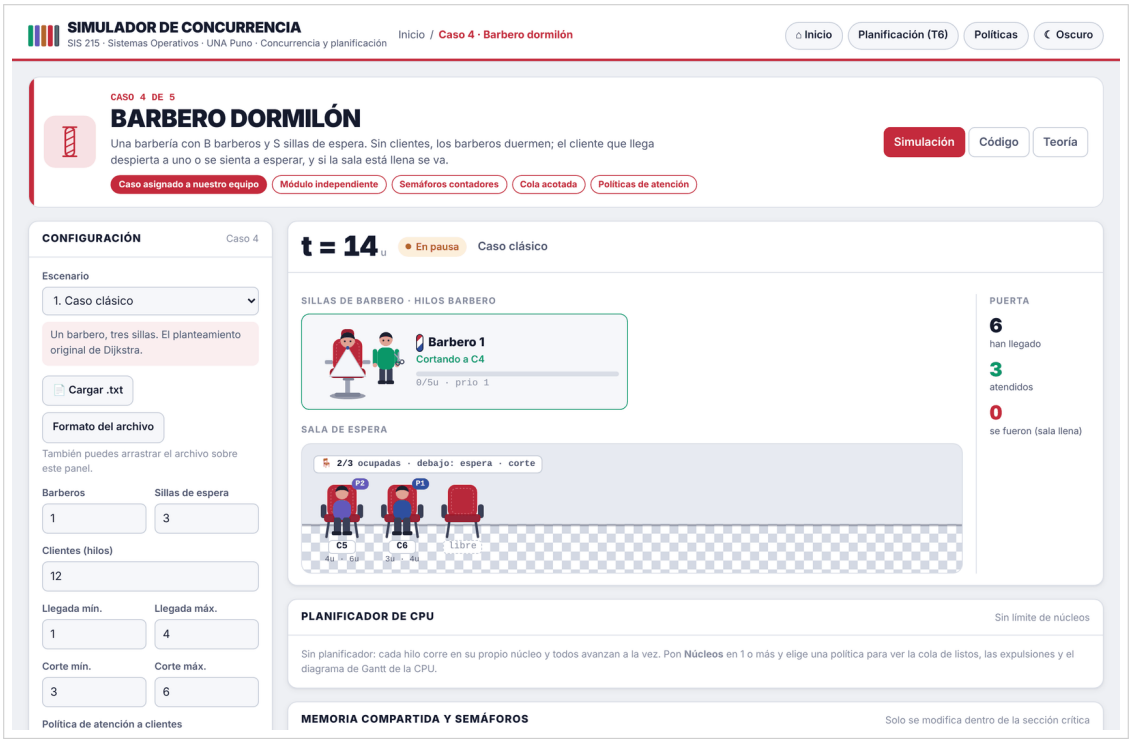
Se usa el «Caso clásico»: 1 barbero, 3 sillas de espera y 12 clientes con cortes de 3 a 6 unidades.

7.4.1 Vista general y configuración

La vista se capturó en $t = 14$: el barbero corta a C4 y dos clientes esperan sentados. A la izquierda está la configuración del caso (Figura 43).

Figura 42

Caso 4, Barbero dormilón: vista general del módulo



Nota. Cabecera «Caso 4 de 5 · Módulo independiente», configuración a la izquierda y escena a la derecha.

Figura 43
Caso 4, Barbero dormilón: panel de configuración

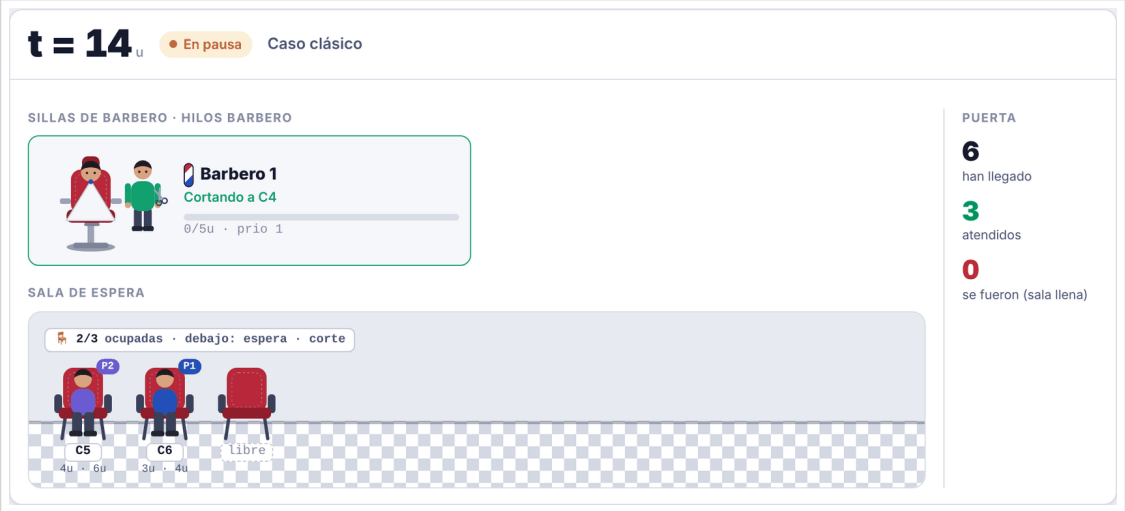


7.4.2 Escena, memoria compartida y métricas

El sillón muestra al barbero cortando (o dormido cuando no hay clientes) y la sala de espera tiene sillas reales; el contador de la puerta indica llegados, atendidos y rechazados. `esperando` = 2 coincide con la sala real (2 clientes); el semáforo `clientes` = 2 y `barberos` = 0 porque el barbero está ocupado.

Figura 44

Caso 4, Barbero dormilón: escena de la solución correcta



Nota. Los colores de los personajes indican su estado.

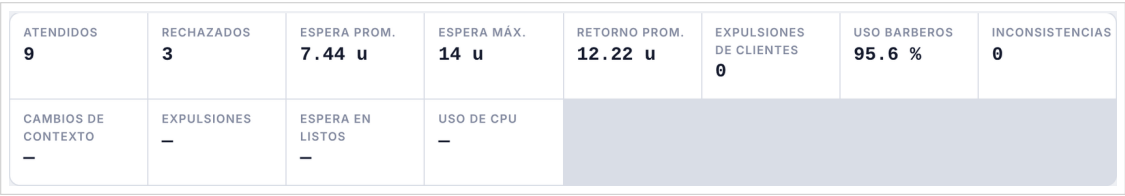
Figura 45

Caso 4, Barbero dormilón: memoria compartida y métricas

(a) Memoria compartida y semáforos



(b) Métricas al terminar el escenario



Nota. Los valores en rojo indicarían una inconsistencia; aquí no hay ninguna.

7.4.3 Hilos y bitácora

La tabla de clientes muestra la llegada, el corte, lo que falta, la prioridad, el estado y el barbero asignado; la bitácora muestra cada `wait(mutex)` y `signal(clientes)`.

Figura 46

Caso 4, Barbero dormilón: tabla de hilos y bitácora de eventos

(a) Tabla de hilos *(b) Bitácora de eventos*



HILOS CLIENTE — Datos privados de cada hilo

HILO	LLEGA	CORTE	RESTA	PRIO.	ESTADO	BARBERO	EXPULS.
C1	0	3u	0u	1	Atendido	B1	–
C2	4	5u	0u	2	Atendido	B1	–
C3	6	4u	0u	1	Atendido	B1	–
C4	9	5u	5u	1	En corte	B1	–
C5	10	6u	6u	2	Esperando	–	–
C6	11	4u	4u	1	Esperando	–	–
C7	14	4u	4u	3	Por llegar	–	–
C8	15	6u	6u	1	Por llegar	–	–
C9	16	3u	3u	1	Por llegar	–	–
C10	19	6u	6u	1	Por llegar	–	–
C11	23	5u	5u	2	Por llegar	–	–

BITÁCORA DE EVENTOS — 16 eventos

```
4   B1 despierta: wait(clientes) ✓ · esperando=1→0 · signal(barberos) → corta a C2 (esperó 0u)
6   C3 llega (corte 4u, prio 1) · wait(mutex) · esperando=0→1 · signal(clientes)
9   B1 termina el corte de C2; el hilo C2 finaliza
9   B1: wait(clientes) ✓ · esperando=1→0 · signal(barberos) → corta a C3 (esperó 3u)
9   C4 llega (corte 5u, prio 1) · wait(mutex) · esperando=0→1 · signal(clientes)
10  C5 llega (corte 6u, prio 2) · wait(mutex) · esperando=1→2 · signal(clientes)
11  C6 llega (corte 4u, prio 1) · wait(mutex) · esperando=2→3 · signal(clientes)
13  B1 termina el corte de C3; el hilo C3 finaliza
13  B1: wait(clientes) ✓ · esperando=3→2 · signal(barberos) → corta a C4 (esperó 4u)
```

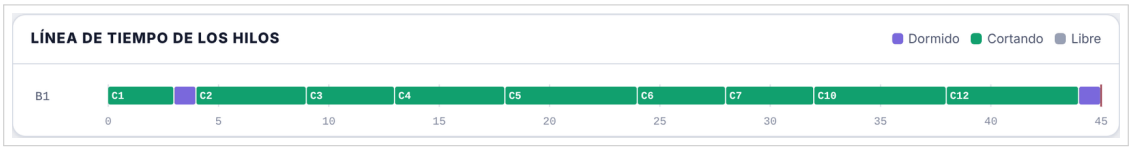
Nota. La tabla guarda los datos privados de cada hilo; la bitácora registra cada operación sobre los semáforos.

7.4.4 Línea de tiempo

Al terminar ($T = 44$ u) se atendieron 9 clientes y 3 se fueron por sala llena, con 95,6 % de uso del barbero.

Figura 47

Caso 4, Barbero dormilón: línea de tiempo de los hilos



Nota. Una fila por hilo y un color por estado; la leyenda está en la parte superior.

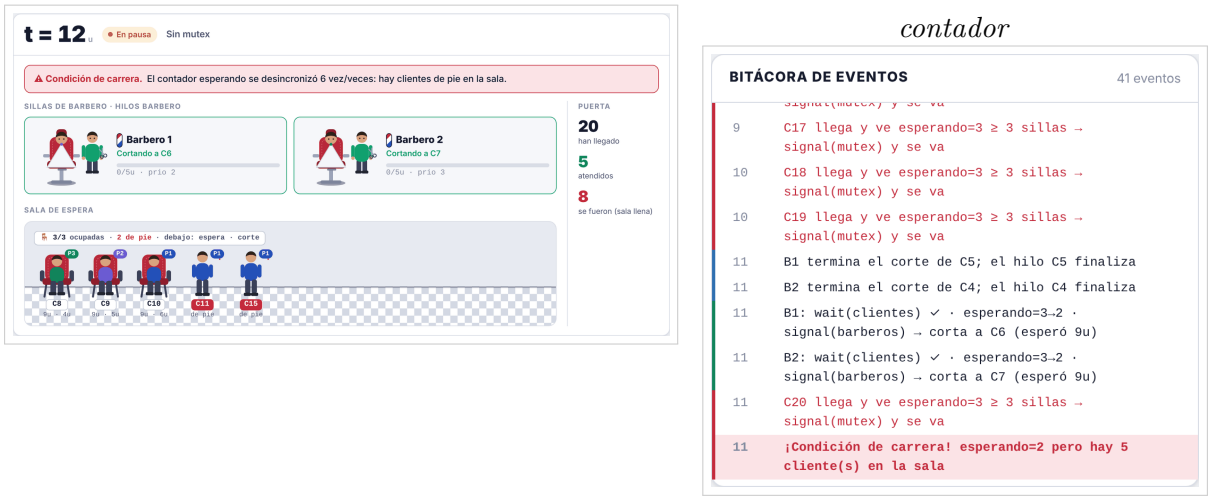
7.4.5 Falla demostrada

Con el escenario «Sin mutex» (2 barberos y 3 sillas) el contador `esperando` pierde actualizaciones y entran clientes que quedan de pie; la bitácora detecta que el contador no coincide con la sala.

Figura 48

Caso 4, Barbero dormilón: falla de la solución incorrecta

(a) Clientes de pie por la condición de carrera *(b) Bitácora con la inconsistencia del contador*



Nota. El simulador detecta la falla automáticamente y la marca en rojo.

7.4.6 Planificador de CPU y comparación de políticas

Con 3 barberos y un solo núcleo, solo un barbero puede cortar en cada unidad; los otros esperan en la cola de listos. En este caso el botón compara las siete *políticas de atención* a clientes: cambian los atendidos, la espera y las expulsiones de clientes.

Figura 49

Caso 4, Barbero dormilón: planificador de CPU



Nota. Núcleos con el hilo que ejecuta, cola de listos en orden de despacho y diagrama de Gantt de la CPU.

Figura 50

Caso 4, Barbero dormilón: comparación de las 7 políticas

HISTORIAL DE ESTE CASO

Ejecutar todos los escenarios | Comparar las 7 políticas de atención | Limpiar

#	ESCENARIO	ATENDIDOS	RECHAZADOS	ESPERA PROM.	ESPERA MÁX.	RETORNO PROM.	EXPULSIONES DE CLIENTES	USO BARBEROS	INCONSIST
1	Comparar · FIFO · orden de llegada	11	9	23.18 u	38 u	28.00 u	0	32.7 %	0
2	Comparar · SJF · corte más corto	11	9	20.64 u	40 u	25.45 u	0	32.7 %	0
3	Comparar · Prioridad (1 = alta)	11	9	23.18 u	38 u	28.00 u	0	32.7 %	0
4	Comparar · HRRN · mayor razón de respuesta	11	9	21.27 u	40 u	26.09 u	0	32.7 %	0
5	Comparar · SRT · menor corte restante	12	8	22.17 u	42 u	27.08 u	2	32.8 %	0
6	Comparar · Round Robin (quantum de atención)	11	9	23.18 u	38 u	28.00 u	0	32.7 %	0
7	Comparar · Prioridad expulsiva	9	11	22.89 u	33 u	27.67 u	3	32.6 %	0

Nota. Cada fila es una ejecución completa con la misma configuración y semilla.

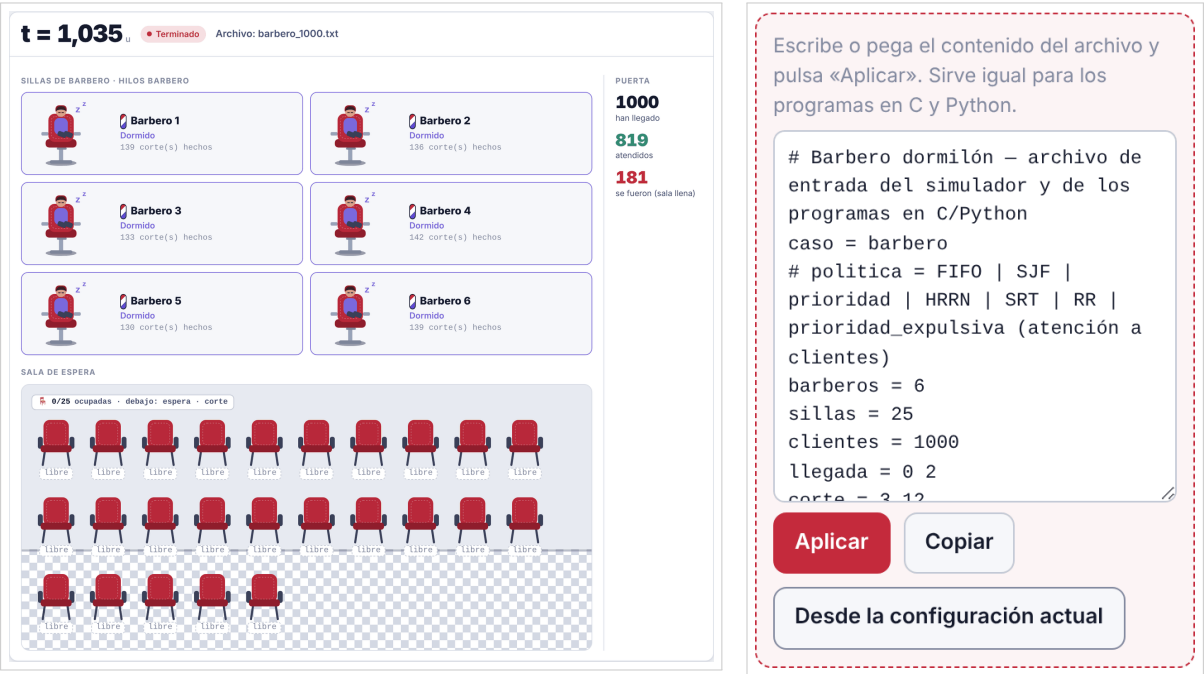
7.4.7 Caso grande cargado desde un archivo

El archivo `barbero_1000.txt` define 6 barberos, 25 sillas y 1 000 clientes con prioridad y envejecimiento. Terminó en $t = 1\,035$: 819 atendidos y 181 que encontraron la sala llena.

Figura 51

Caso 4, Barbero dormilón: ejecución con un archivo de entrada grande

(a) Escena al terminar *(b) Formato del archivo*



Nota. Con miles de hilos la escena dibuja los primeros y resume el resto. El mismo archivo sirve para los programas en C y Python.

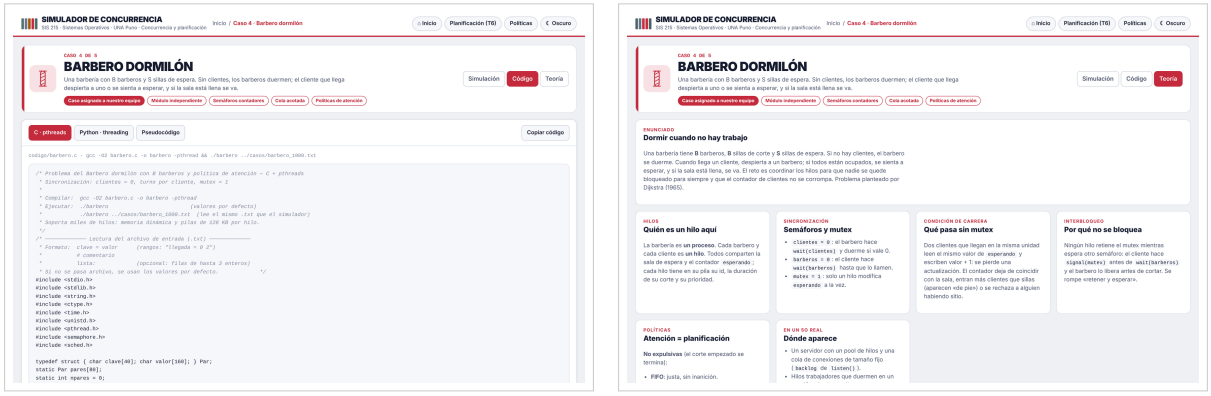
7.4.8 Código y teoría

La pestaña «Código» muestra el programa en C, en Python y el pseudocódigo del caso; la pestaña «Teoría» explica el problema, la solución, qué pasa sin sincronización y el formato del archivo.

Figura 52

Caso 4, Barbero dormilón: pestañas de código y teoría

(a) Código *(b) Teoría*



Nota. El código es el mismo que está en la carpeta `codigo` del repositorio.

7.4.9 Resultados de los escenarios

La Tabla 8 resume los 15 escenarios del caso.

Tabla 8
Resultados de los escenarios del caso Barbero dormilón

Escenario	T	Atendidos	Rechazados	Espera prom.	Espera máx.	Retorno prom.	Uso barberos	Inconsistencias
Caso clásico	44	9	3	7.44 u	14 u	12.22 u	95.6 %	0
Sala saturada	29	6	14	6.83 u	10 u	11.67 u	96.7 %	0
Barbería grande	54	23	7	6.87 u	12 u	13.30 u	89.7 %	0
Barberos ociosos	66	10	0	0.00 u	0 u	3.00 u	22.4 %	0
Sin sillas de espera	26	8	7	0.00 u	0 u	4.88 u	72.2 %	0
SJF	94	16	9	23.38 u	63 u	29.25 u	98.9 %	0
Prioridad sin envejecimiento	74	18	7	18.33 u	51 u	22.44 u	98.7 %	0
Prioridad con envejecimiento	75	19	6	17.84 u	32 u	21.79 u	98.7 %	0
Sin mutex	32	13	12	9.38 u	18 u	14.15 u	93.9 %	7
Atención HRRN	86	15	10	22.53 u	53 u	28.27 u	98.9 %	0
Atención SRT (expulsiva)	85	16	9	20.38 u	64 u	25.69 u	98.8 %	0
Atención Round Robin q=3	65	12	13	28.67 u	44 u	34.08 u	98.5 %	0
Atención prioridad expulsiva	75	19	6	17.95 u	32 u	21.89 u	98.7 %	0
CPU · 3 barberos y 1 núcleo	53	11	9	23.18 u	38 u	28.00 u	32.7 %	0
Mil clientes	1035	819	181	26.85 u	40 u	34.39 u	99.3 %	0

Nota. T es el tiempo total en unidades. Inconsistencias cuenta las veces en que el contador `esperando` no coincidió con la sala real. En rojo, los fallos de sincronización.

La Tabla 9 compara las siete políticas de atención con la misma carga (1 barbero, 8 sillas y 25 clientes con cortes de 2 a 10 unidades). SRT logra la menor espera promedio y el menor tiempo de retorno, pero su espera máxima es la más alta: los cortes largos son adelantados una y otra vez. SJF y Prioridad muestran el mismo efecto sin expulsión. HRRN reduce la espera máxima respecto de SJF sin perder la ventaja en el promedio, y Round Robin iguala la espera máxima de FIFO a costa de un tiempo de retorno mayor.

Tabla 9

Comparación de las siete políticas de atención a clientes del barbero

Política de atención	Clientes atendidos	Clientes rechazados	Espera prom.	Espera máx.	Retorno prom.	Expulsiones
FIFO: orden de llegada	14	11	23.93	44	29.36	0
SJF: corte más corto	16	9	23.38	63	29.25	0
Prioridad (1 = alta)	15	10	23.27	59	28.80	0
HRRN: mayor razón de respuesta	15	10	22.53	53	28.27	0
SRT: menor corte restante ⋆	16	9	**20.38**	64	**25.69**	2
Round Robin (quantum de atención) ⋆	12	13	28.67	44	34.08	14
Prioridad expulsiva ⋆	13	12	23.85	51	29.62	2

Nota. ⋆ = política expulsiva. Carga: 1 barbero, 8 sillas, 25 clientes, cortes de 2 a 10 unidades y quantum de atención de 3. Tiempos en unidades; en negrita, el mejor valor.

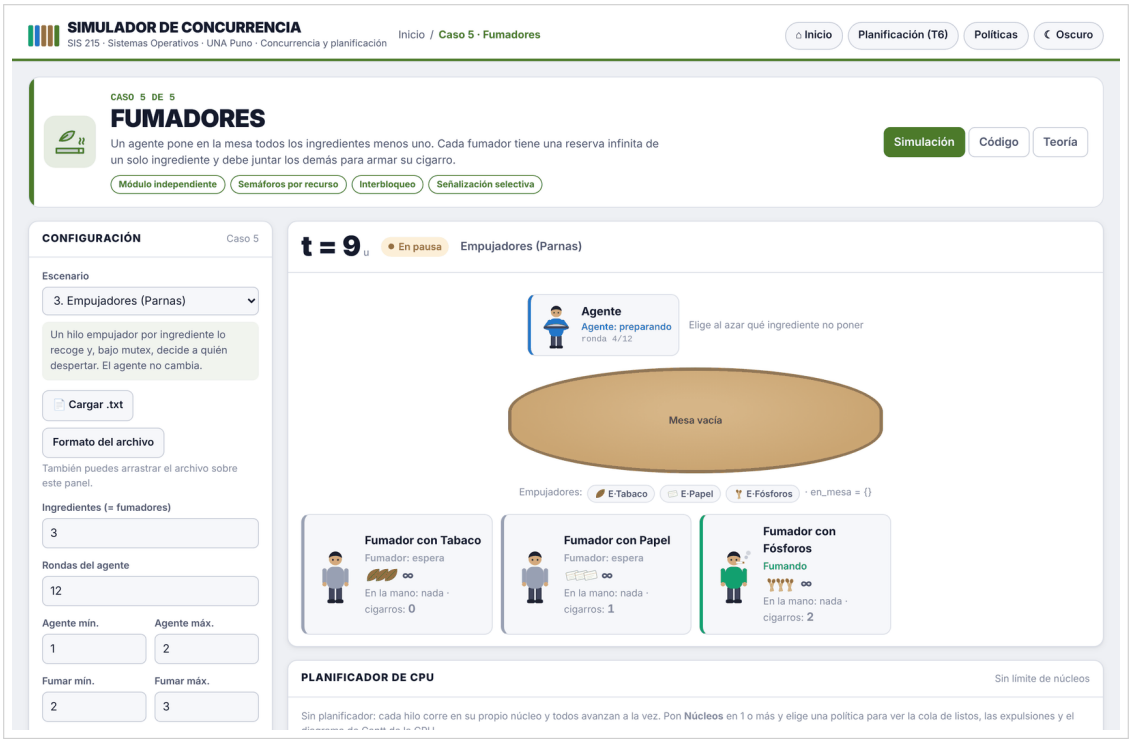
7.5 Caso 5: Fumadores

Se usa el escenario «Empujadores (Parnas)»: 3 fumadores, 3 ingredientes y 12 rondas del agente.

7.5.1 Vista general y configuración

La vista se capturó en $t = 9$: el fumador con fósforos fuma mientras el agente prepara la siguiente ronda. A la izquierda está la configuración del caso (Figura 54).

Figura 53
Caso 5, Fumadores: vista general del módulo



Nota. Cabecera «Caso 5 de 5 · Módulo independiente», configuración a la izquierda y escena a la derecha.

Figura 54

Caso 5, Fumadores: panel de configuración



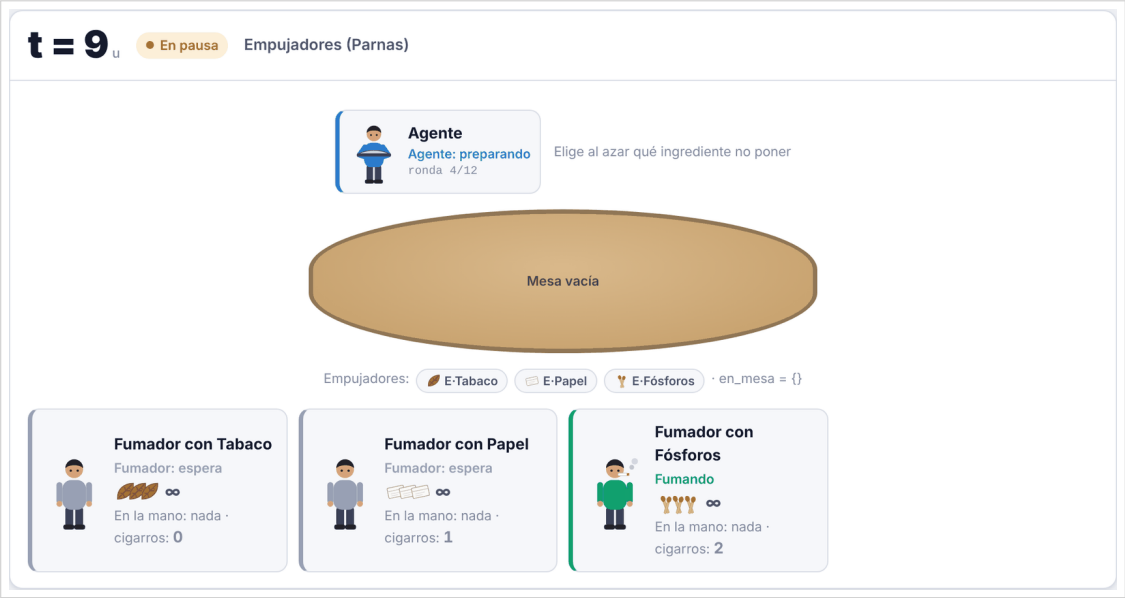
Nota. Escenario, parámetros, carga de archivo .txt, planificador de CPU y controles de ejecución.

7.5.2 Escena, memoria compartida y métricas

El agente está sobre la mesa de madera; cada fumador muestra su reserva infinita, lo que tiene en la mano y sus cigarros, y suelta humo mientras fuma. Los semáforos de los ingredientes están en 0 porque los empujadores ya recogieron la ronda; `mutex` = 1 protege lo que hay en la mesa.

Figura 55

Caso 5, Fumadores: escena de la solución correcta



Nota. Los colores de los personajes indican su estado.

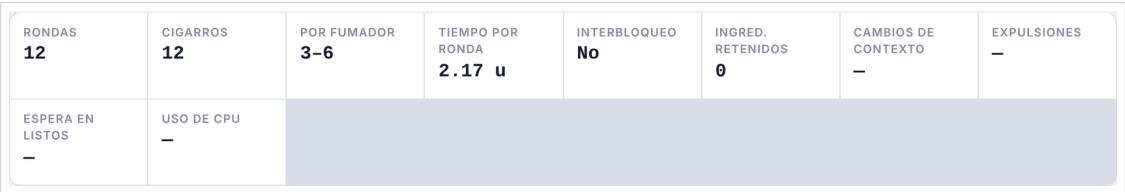
Figura 56

Caso 5, Fumadores: memoria compartida y métricas

(a) Memoria compartida y semáforos



(b) Métricas al terminar el escenario



Nota. Los valores en rojo indicarían una inconsistencia; aquí no hay ninguna.

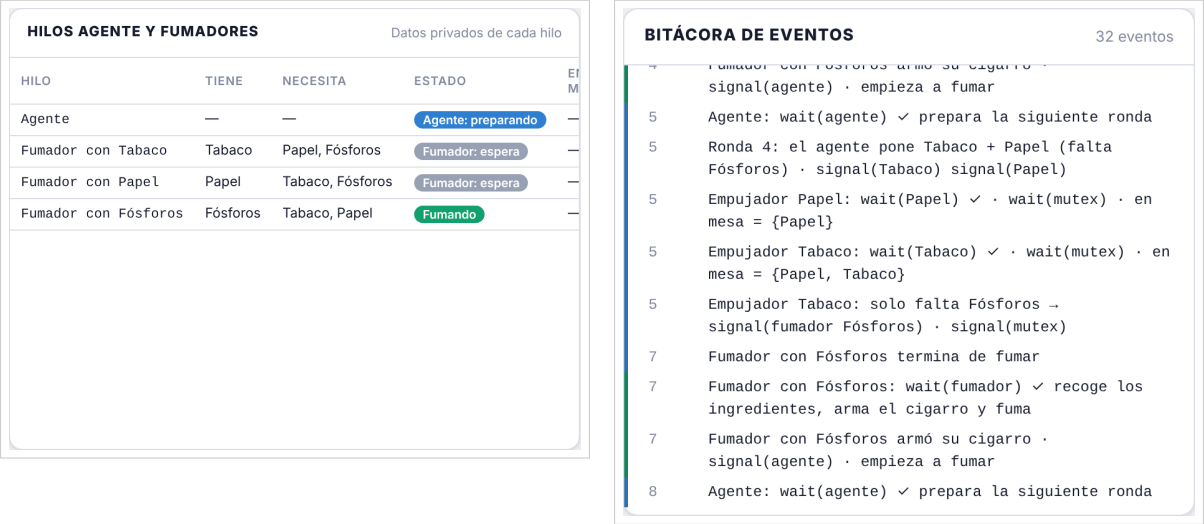
7.5.3 Hilos y bitácora

La tabla indica qué tiene y qué necesita cada fumador; la bitácora muestra a los empujadores anotando los ingredientes y despertando solo al fumador que puede fumar.

Figura 57

Caso 5, Fumadores: tabla de hilos y bitácora de eventos

(a) Tabla de hilos *(b) Bitácora de eventos*



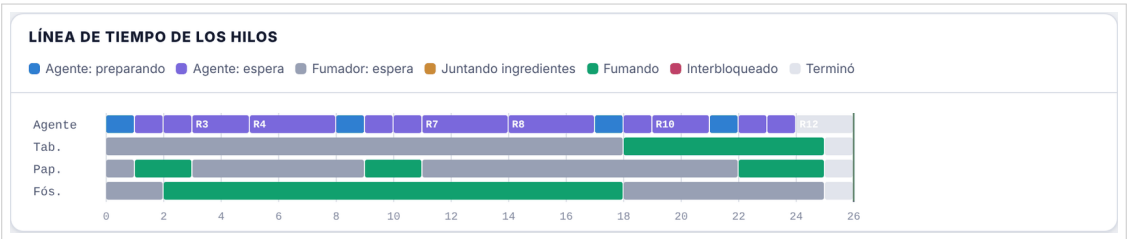
Nota. La tabla guarda los datos privados de cada hilo; la bitácora registra cada operación sobre los semáforos.

7.5.4 Línea de tiempo

Al terminar ($T = 25$ u) se completaron las 12 rondas sin interbloqueo.

Figura 58

Caso 5, Fumadores: línea de tiempo de los hilos



Nota. Una fila por hilo y un color por estado; la leyenda está en la parte superior.

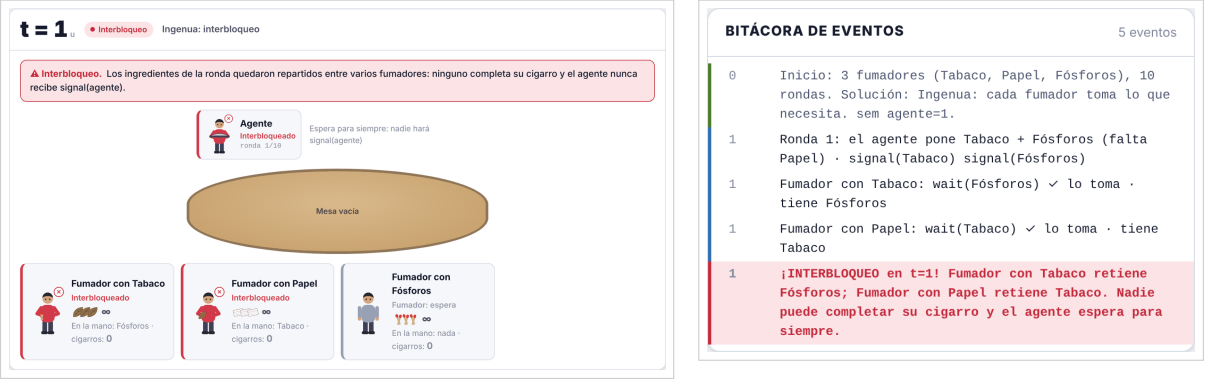
7.5.5 Falla demostrada

Con la solución ingenua, en la primera ronda el fumador con tabaco toma los fósforos y el fumador con papel toma el tabaco: ninguno completa su cigarro y el agente espera para siempre.

Figura 59

Caso 5, Fumadores: falla de la solución incorrecta

(a) Mesa con el interbloqueo *(b) Bitácora del interbloqueo*



Nota. El simulador detecta la falla automáticamente y la marca en rojo.

7.5.6 Planificador de CPU y comparación de políticas

Con un núcleo y Round Robin, el agente y los fumadores compiten por la CPU para preparar la ronda y fumar. Las siete políticas completan las 10 rondas; las expulsivas generan más cambios de contexto.

Figura 60

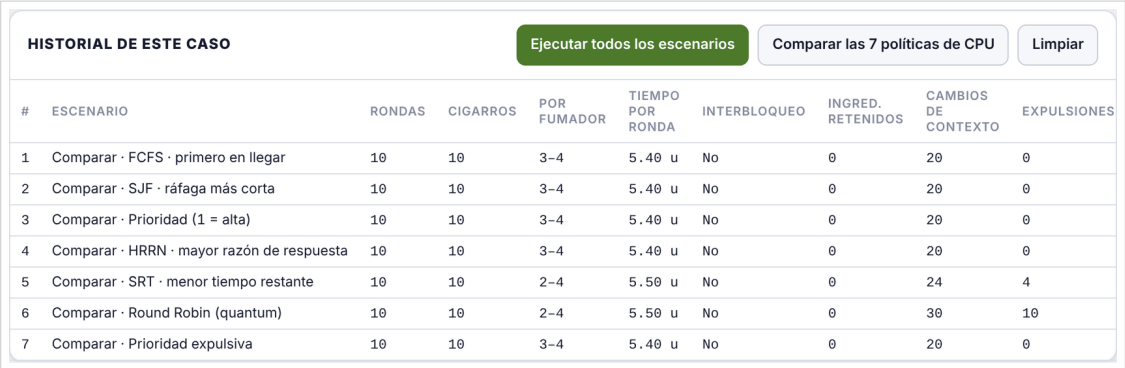
Caso 5, Fumadores: planificador de CPU



Nota. Núcleos con el hilo que ejecuta, cola de listos en orden de despacho y diagrama de Gantt de la CPU.

Figura 61

Caso 5, Fumadores: comparación de las 7 políticas



HISTORIAL DE ESTE CASO

Ejecutar todos los escenarios | Comparar las 7 políticas de CPU | Limpiar

#	ESCENARIO	RONDAS	CIGARROS	POR FUMADOR	TIEMPO POR RONDA	INTERBLOQUEO	INGRED. RETENIDOS	CAMBIOS DE CONTEXTO	EXPULSIONES
1	Comparar · FCFS · primero en llegar	10	10	3–4	5.40 u	No	0	20	0
2	Comparar · SJF · ráfaga más corta	10	10	3–4	5.40 u	No	0	20	0
3	Comparar · Prioridad (1 = alta)	10	10	3–4	5.40 u	No	0	20	0
4	Comparar · HRRN · mayor razón de respuesta	10	10	3–4	5.40 u	No	0	20	0
5	Comparar · SRT · menor tiempo restante	10	10	2–4	5.50 u	No	0	24	4
6	Comparar · Round Robin (quantum)	10	10	2–4	5.50 u	No	0	30	10
7	Comparar · Prioridad expulsiva	10	10	3–4	5.40 u	No	0	20	0

Nota. Cada fila es una ejecución completa con la misma configuración y semilla.

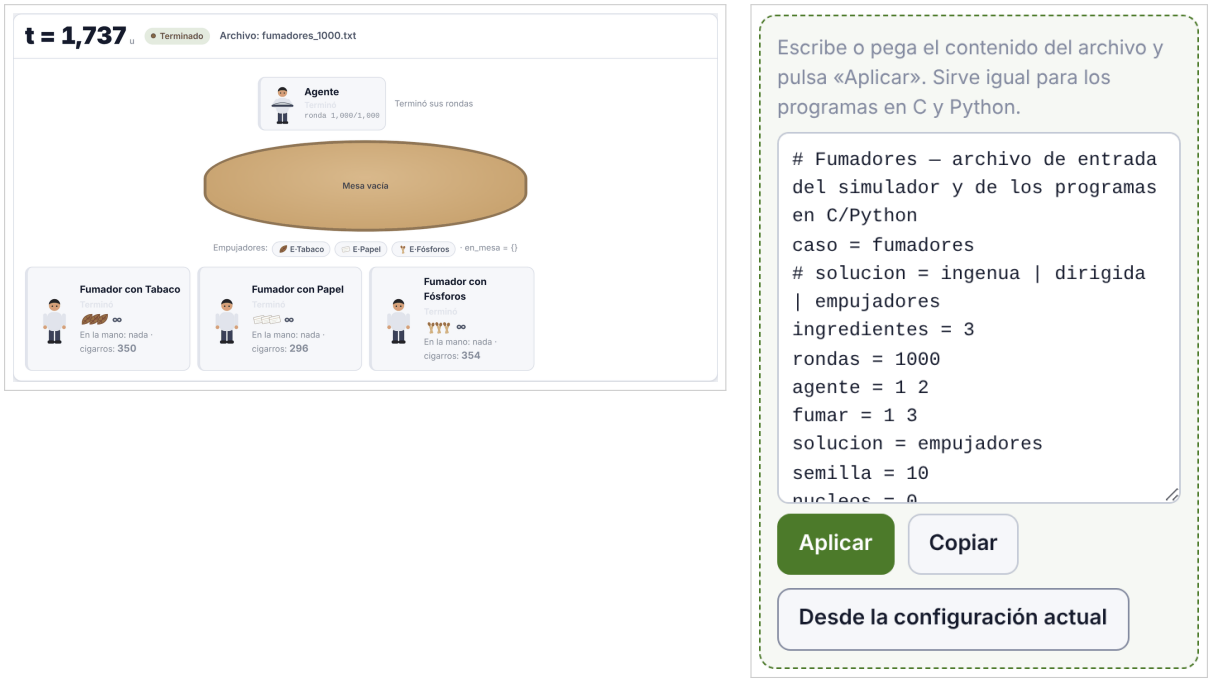
7.5.7 Caso grande cargado desde un archivo

El archivo `fumadores_1000.txt` define 1 000 rondas con la solución de empujadores. Terminó en $t = 1\,737$ sin interbloqueo.

Figura 62

Caso 5, Fumadores: ejecución con un archivo de entrada grande

(a) Escena al terminar *(b) Formato del archivo*



Nota. Con miles de hilos la escena dibuja los primeros y resume el resto. El mismo archivo sirve para los programas en C y Python.

7.5.8 Código y teoría

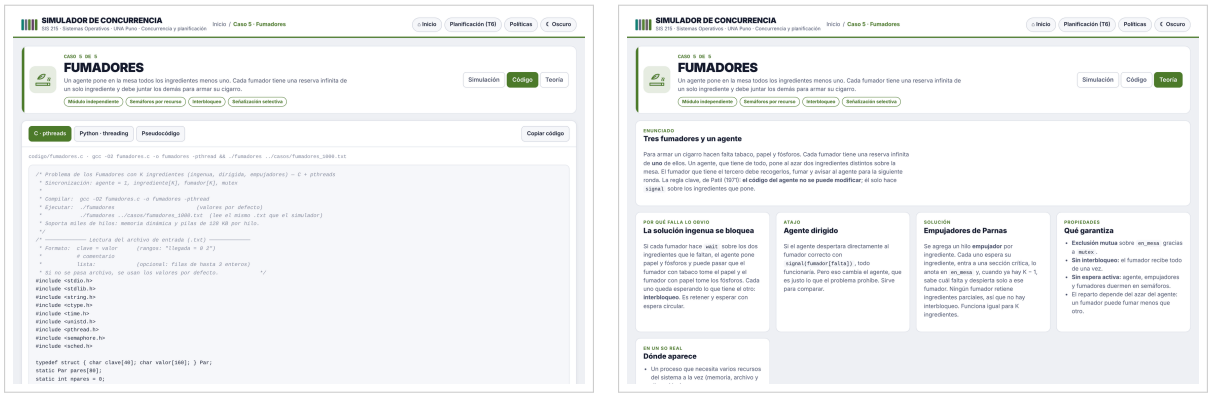
La pestaña «Código» muestra el programa en C, en Python y el pseudocódigo del caso; la pestaña «Teoría» explica el problema, la solución, qué pasa sin sincronización y

el formato del archivo.

Figura 63

Caso 5, Fumadores: pestañas de código y teoría

(a) Código *(b) Teoría*



Nota. El código es el mismo que está en la carpeta `codigo` del repositorio.

7.5.9 Resultados de los escenarios

La Tabla 10 resume los 9 escenarios del caso.

Tabla 10

Resultados de los escenarios del caso Fumadores

Escenario	T	Rondas	Cigarros	Por fumador	Tiempo por ronda	Interbloqueo	Ingr. retenidos
Ingenua: interbloqueo	1	1	0	0–0	2.00 u	**Sí (t=1)**	**2**
Agente dirigido	21	10	10	1–6	2.20 u	No	0
Empujadores (Parnas)	25	12	12	3–6	2.17 u	No	0
Cinco ingredientes	27	15	15	2–4	1.87 u	No	0
CPU · 1 núcleo prioridad	53	10	10	3–4	5.40 u	No	0
CPU · 1 núcleo Round Robin	54	10	10	2–4	5.50 u	No	0
CPU · 1 núcleo SRT	54	10	10	2–4	5.50 u	No	0
Mil rondas	1737	1000	1000	296–354	1.74 u	No	0
Ingenua con 4 ingredientes	1	1	0	0–0	2.00 u	**Sí (t=1)**	**3**

Nota. T es el tiempo total en unidades. «Por fumador» indica el mínimo y el máximo de cigarros que fumó cada fumador. En rojo, los fallos de sincronización.

8. Módulo 6: simulador de planificación de procesos (Tarea 6)

El módulo 6 es un simulador del planificador de corto plazo. Se entra desde la tarjeta «Planificación de procesos» de la pantalla de inicio o desde el botón «Planificación (T6)» de la barra superior (Figura 64). Como los otros módulos, es independiente: tiene su propia configuración, su reloj y su comparación de políticas. Su motor simula una unidad de tiempo por vuelta y es el mismo algoritmo de los programas `planificacion.c` y `planificacion.py`.

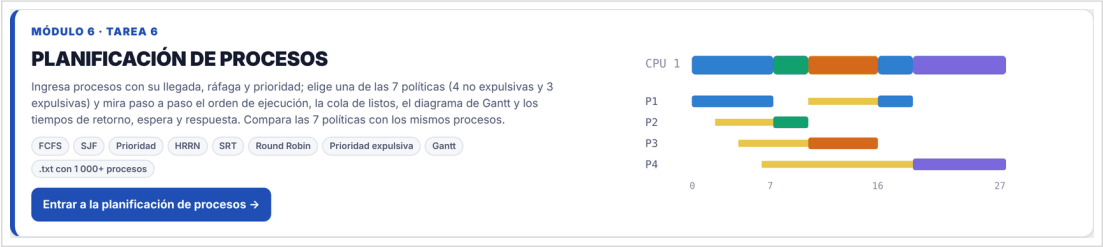
8.1 Requisitos de la Tarea 6 y cómo se cumplen

Tabla 11
Requisitos de la Tarea 6 y su implementación

Requisito	Implementación	Evidencia
Ingresar procesos con sus datos	Tabla editable con nombre, llegada, ráfaga y prioridad; agregar, quitar, generar al azar o cargar un .txt.	Fig. 66, 67
Políticas no expulsivas	FCFS, SJF, Prioridad y HRRN.	Fig. 75
Políticas expulsivas	SRT, Round Robin (quantum configurable) y Prioridad expulsiva.	Fig. 76
Orden de ejecución	Secuencia de tramos con proceso, inicio y fin; cola de listos en orden de despacho.	Fig. 69, 70
Calcular los resultados	Inicio, fin, retorno, espera, respuesta y expulsiones por proceso; promedios, rendimiento, uso de CPU y cambios de contexto.	Fig. 72, Tabla 13
Diagrama de Gantt	Gantt por CPU que avanza con el reloj, y diagrama de estados por proceso.	Fig. 70, 71
Interfaz gráfica	Aplicación web con controles de reproducción, temas claro y oscuro y vista para celular.	Fig. 65, 82
Visualizar cómo trabaja el SO	Bitácora del planificador y comparación de las 7 políticas con los mismos procesos.	Fig. 74, 77

Figura 64

Acceso al módulo 6 desde la pantalla de inicio



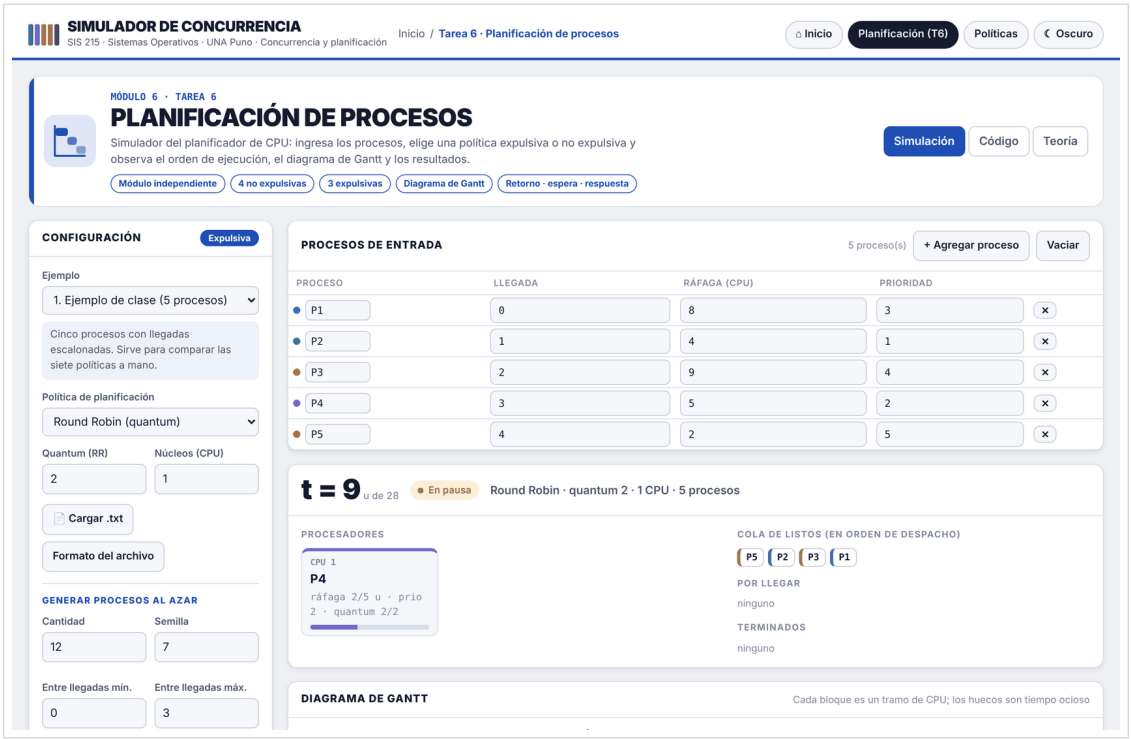
Nota. La tarjeta resume las políticas disponibles y abre el simulador de planificación.

8.2 Vista general, ingreso de procesos y configuración

La vista general (Figura 65) tiene a la izquierda la configuración y a la derecha la tabla de procesos de entrada, el estado de la CPU, el diagrama de Gantt, el orden de ejecución, los estados, los resultados, la bitácora y la comparación de políticas. Los procesos se escriben directamente en la tabla (Figura 66); cada cambio vuelve a calcular la planificación. La configuración (Figura 67) incluye 9 ejemplos, la política, el quantum, el número de CPU, la carga de archivos y un generador de procesos aleatorios.

Figura 65

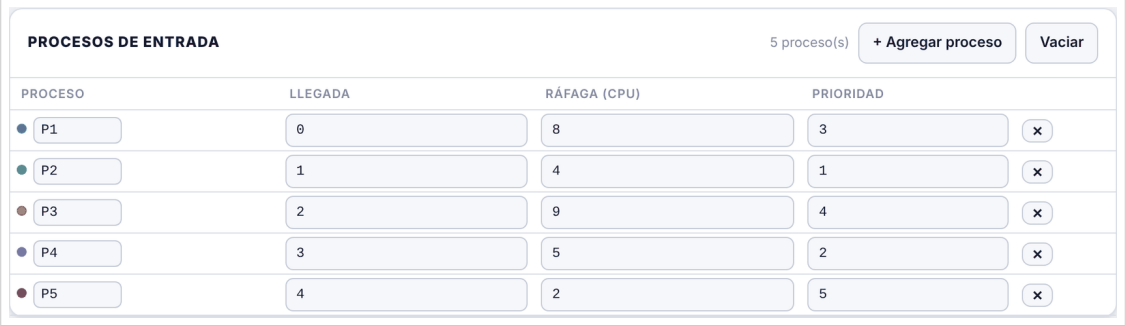
Módulo 6: vista general con Round Robin en $t = 9$



Nota. Ejemplo de clase con cinco procesos y quantum 2.

Figura 66

Tabla de procesos de entrada



Nota. Cada fila es un proceso; el punto de color es el que usa en el Gantt.

Figura 67

Panel de configuración del módulo 6



Nota. La etiqueta superior indica si la política elegida es expulsiva.

Figura 68

Archivo de entrada del módulo 6

(a) Formato del archivo

Una fila por proceso: nombre llegada ráfaga prioridad. Sirve igual para los programas en C y Python.

```
# Planificación de procesos — archivo de entrada del simulador y de los programas en C/Python
caso = planificacion
politica = fcfs   # fcfs | sjf | prioridad | hrrn | srt | rr | prioridad_expulsiva
quantum = 2            # solo Round Robin
nucleos = 1            # CPU disponibles (1 a 8)
# Para generar procesos al azar, en lugar de la lista:
```

Aplicar | Copiar

Desde la tabla actual

(b) Errores de validación

CONFIGURACIÓN No expulsiva

Ejemplo

1. Ejemplo de clase (5 procesos)

Cinco procesos con llegadas escalonadas. Sirve para comparar las siete políticas a mano.

Política de planificación

FCFS · primero en llegar, primero e

Quantum (RR) 2

Núcleos (CPU) 1

Cargar .txt

Formato del archivo

Una fila por proceso: nombre llegada ráfaga prioridad. Sirve igual para los programas en C y Python.

```
caso = planificacion
politica = lottery
quantum = 0
lista:
P1 0 x 2
P2 3 0 1
```

Aplicar | Copiar

Desde la tabla actual

No se pudo cargar el archivo:

- Línea 5: cada fila lleva [nombre] llegada ráfaga [prioridad] con números enteros («P1 0 x 2»).
- Línea 6: la ráfaga debe ser al menos 1.
- Línea 2: «lottery» no es una política. Usa: fcfs, sjf, prioridad, hrrn, srt, rr, round_robin, prioridad_expulsiva.
- Línea 3: «quantum» debe ser un

8.3 Ejecución paso a paso

Con «Paso» el reloj avanza una unidad; con «Iniciar» avanza a la velocidad elegida. En $t = 9$ con Round Robin (Figura 69), P4 está en la CPU con 2 de 5 unidades hechas y su quantum en 2 de 2, y la cola de listos tiene P5, P2, P3 y P1 en el orden en que serán despachados. El Gantt y el orden de ejecución crecen junto con el reloj (Figura 70); en el diagrama de estados, la franja amarilla es el tiempo que cada proceso pasa listo esperando la CPU (Figura 71).

Figura 69

Estado de la CPU y de las colas en $t = 9$

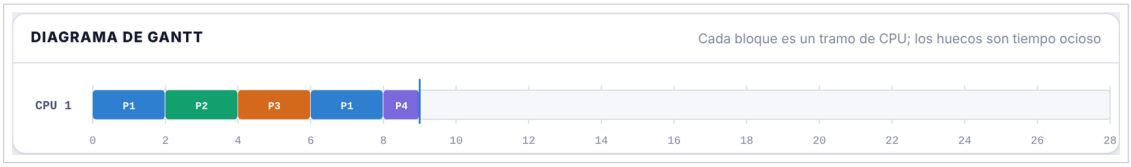


Nota. Procesadores, cola de listos en orden de despacho, procesos por llegar y terminados.

Figura 70

Diagrama de Gantt y orden de ejecución a mitad de la simulación

(a) Diagrama de Gantt



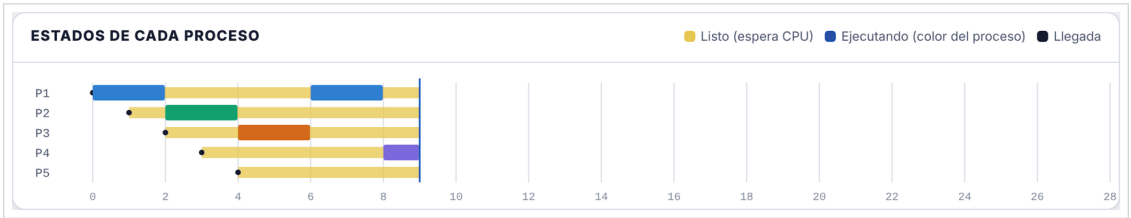
(b) Orden de ejecución



Nota. La línea vertical marca el instante actual.

Figura 71

Diagrama de estados de cada proceso



Nota. Punto negro: llegada; franja amarilla: listo; bloque de color: ejecutando.

8.4 Resultados por proceso

Al terminar, la tabla de resultados muestra para cada proceso su inicio, fin, retorno, espera, respuesta y expulsiones, con los promedios (Figura 72), y las tarjetas resumen el rendimiento, el uso de CPU y los cambios de contexto. La bitácora explica cada decisión del planificador (Figura 74).

Figura 72

Tabla de resultados durante y al final de la simulación (Round Robin)

(a) En $t = 9$

TABLA DE RESULTADOS

Retorno = fin − llegada · Espera = retorno − ráfaga · Respuesta = inicio − llegada

PROCESO	LLEGADA	RÁFAGA	PRIO.	INICIO	FIN	RETORNO	ESPERA	RESPUESTA	EXPULS.	ESTADO
P1	0	8	3	0	—	—	—	0	—	Listo (en cola)
P2	1	4	1	2	—	—	—	1	—	Listo (en cola)
P3	2	9	4	4	—	—	—	2	—	Listo (en cola)
P4	3	5	2	8	—	—	—	5	—	Ejecutando
P5	4	2	5	—	—	—	—	—	—	Listo (en cola)
Promedio (0 terminados)						—	—	—		

(b) Al terminar

TABLA DE RESULTADOS

Retorno = fin − llegada · Espera = retorno − ráfaga · Respuesta = inicio − llegada

PROCESO	LLEGADA	RÁFAGA	PRIO.	INICIO	FIN	RETORNO	ESPERA	RESPUESTA	EXPULS.	ESTADO
P1	0	8	3	0	24	**24**	**16**	0	3	Terminado
P2	1	4	1	2	14	**13**	**9**	1	1	Terminado
P3	2	9	4	4	28	**26**	**17**	2	3	Terminado
P4	3	5	2	8	25	**22**	**17**	5	2	Terminado
P5	4	2	5	10	12	**8**	**6**	6	0	Terminado
Promedio						**18.60**	**13**	**2.80**		

Nota. Los valores aparecen cuando cada proceso termina.

Figura 73

Indicadores globales de la planificación con Round Robin

TIEMPO TOTAL	RETORNO PROMEDIO	ESPERA PROMEDIO	RESPUESTA PROMEDIO	ESPERA MÁXIMA	RENDIMIENTO	USO DE CPU	CAMBIOS DE CONTEXTO
28 u	18.60 u	13.00 u	2.80 u	17 u	0.179 proc/u	100.0 %	14
EXPULSIONES							
9							

Nota. Tiempo total, promedios, rendimiento, uso de CPU, cambios de contexto y expulsiones.

Figura 74
Bitácora del planificador



Nota. Llegadas, despachos, expulsiones por quantum y fin de cada proceso con su cálculo.

Verificación manual con FCFS. Con el ejemplo de clase (P1: 0, 8, 3; P2: 1, 4, 1; P3: 2, 9, 4; P4: 3, 5, 2; P5: 4, 2, 5 como llegada, ráfaga y prioridad), FCFS ejecuta P1 (0–8), P2 (8–12), P3 (12–21), P4 (21–26) y P5 (26–28). Para P2: retorno $= 12 - 1 = 11$ y espera $= 11 - 4 = 7$. Las esperas son 0, 7, 10, 18 y 22, con promedio $57/5 = 11{,}40$ u, que es el valor que calcula el simulador.

8.5 Las siete políticas con el mismo ejemplo

Las Figuras 75 y 76 muestran el Gantt final de cada política con los mismos cinco procesos, y las Tablas 12 y 13 resumen el orden de ejecución y la espera de cada proceso.

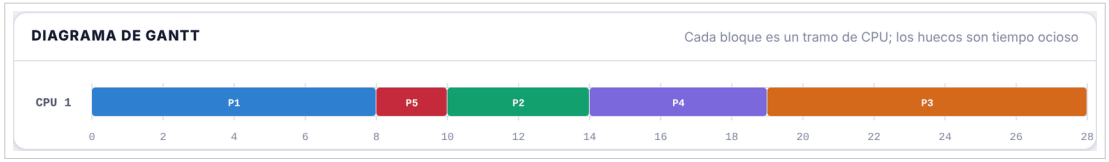
Figura 75

Diagrama de Gantt de las políticas no expulsivas

(a) FCFS



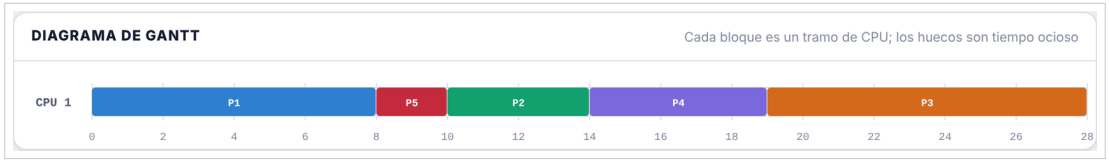
(b) SJF



(c) Prioridad



(d) HRRN

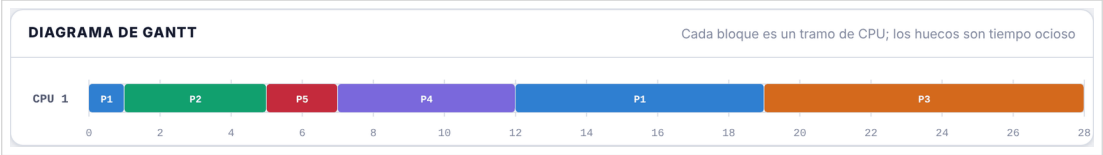


Nota. Ejemplo de clase con 1 CPU. Ningún proceso se interrumpe: cada bloque es una ráfaga completa.

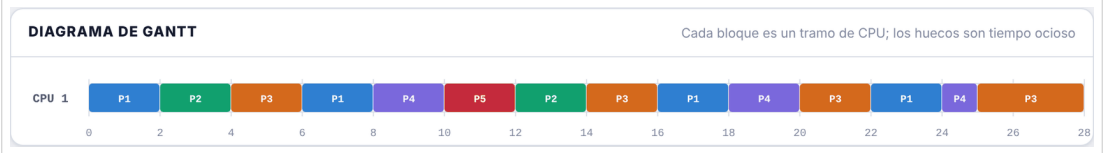
Figura 76

Diagrama de Gantt de las políticas expulsivas

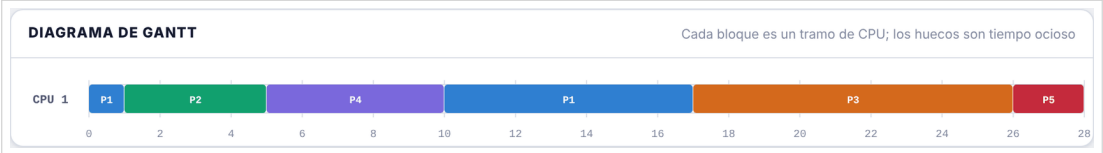
(a) SRT



(b) Round Robin, quantum 2



(c) Prioridad expulsiva



Nota. Ejemplo de clase con 1 CPU. Un mismo proceso aparece en varios bloques cuando fue expulsado.

Tabla 12

Orden de ejecución del ejemplo de clase con cada política

Política	**Orden de ejecución: proceso (inicio–fin)**
FCFS	P1 (0–8) → P2 (8–12) → P3 (12–21) → P4 (21–26) → P5 (26–28)
SJF	P1 (0–8) → P5 (8–10) → P2 (10–14) → P4 (14–19) → P3 (19–28)
Prioridad	P1 (0–8) → P2 (8–12) → P4 (12–17) → P3 (17–26) → P5 (26–28)
HRRN	P1 (0–8) → P5 (8–10) → P2 (10–14) → P4 (14–19) → P3 (19–28)
SRT ⋆	P1 (0–1) → P2 (1–5) → P5 (5–7) → P4 (7–12) → P1 (12–19) → P3 (19–28)
Round Robin ⋆	P1 (0–2) → P2 (2–4) → P3 (4–6) → P1 (6–8) → P4 (8–10) → P5 (10–12) → P2 (12–14) → P3 (14–16) → P1 (16–18) → P4 (18–20) → P3 (20–22) → P1 (22–24) → P4 (24–25) → P3 (25–28)
Prioridad expulsiva ⋆	P1 (0–1) → P2 (1–5) → P4 (5–10) → P1 (10–17) → P3 (17–26) → P5 (26–28)

Nota. ⋆ = política expulsiva. Quantum 2 para Round Robin.

Tabla 13
Tiempo de espera de cada proceso y promedios del ejemplo de clase

Política	P1	P2	P3	P4	P5	Espera prom.	Retorno prom.	Respuesta prom.
FCFS	0	7	10	18	22	11.40	17.00	11.40
SJF	0	9	17	11	4	8.20	13.80	8.20
Prioridad	0	7	15	9	22	10.60	16.20	10.60
HRRN	0	9	17	11	4	8.20	13.80	8.20
SRT ⋆	11	0	17	4	1	6.60	12.20	4.40
Round Robin ⋆	16	9	17	17	6	13.00	18.60	2.80
Prioridad expulsiva ⋆	9	0	15	2	22	9.60	15.20	7.80

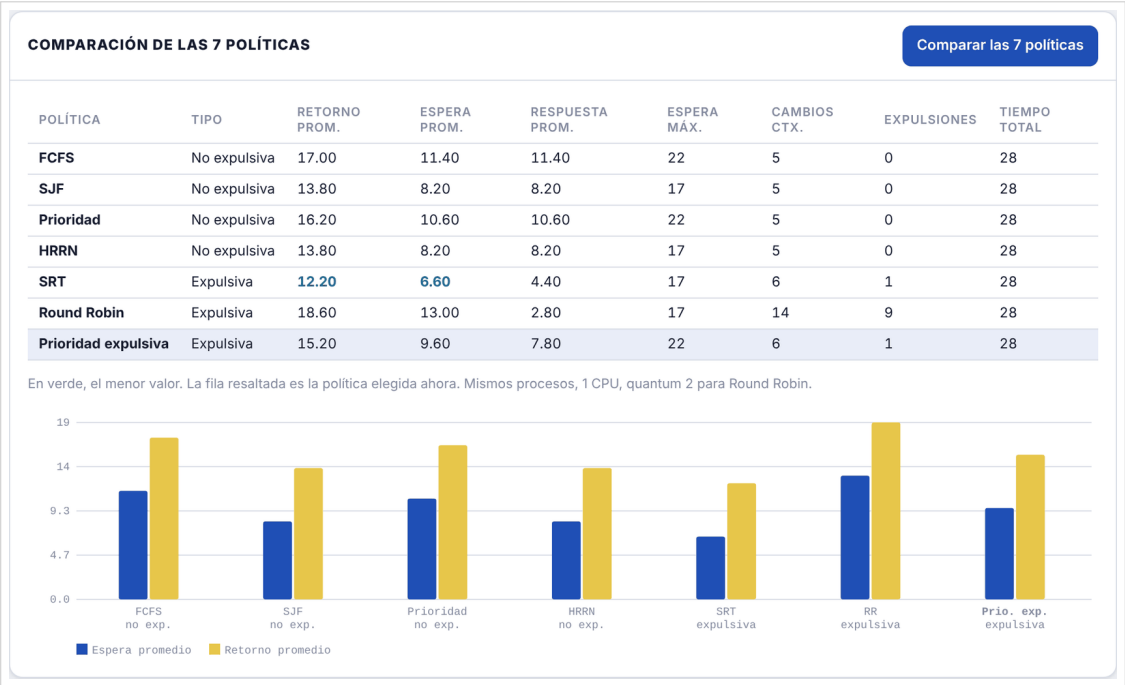
Nota. ⋆ = política expulsiva. Tiempos en unidades. Los mismos valores se obtienen con `planificacion.c` y `planificacion.py`.

SRT logra la menor espera promedio (6,60 u) porque siempre atiende al proceso al que le falta menos. SJF y HRRN coinciden en este ejemplo y mejoran a FCFS en 3,2 u. Round Robin da la menor respuesta promedio (2,80 u), porque todos empiezan pronto, pero la mayor espera (13,00 u) y 14 cambios de contexto. Con Prioridad, P5 (prioridad 5) espera 22 u.

8.6 Comparación de las siete políticas

El botón «Comparar las 7 políticas» ejecuta los mismos procesos con cada política y marca en verde el mejor valor (Figura 77).

Figura 77
Comparación de las siete políticas con el ejemplo de clase



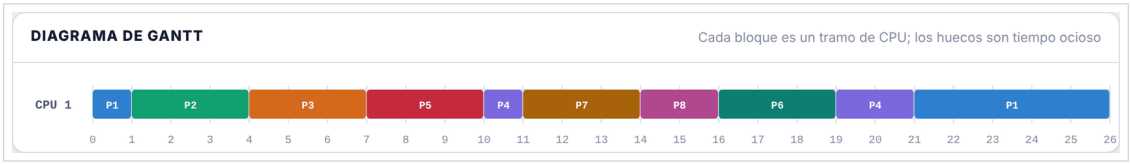
Nota. Tabla de promedios y gráfico de barras de espera y retorno promedio.

8.7 Situaciones especiales

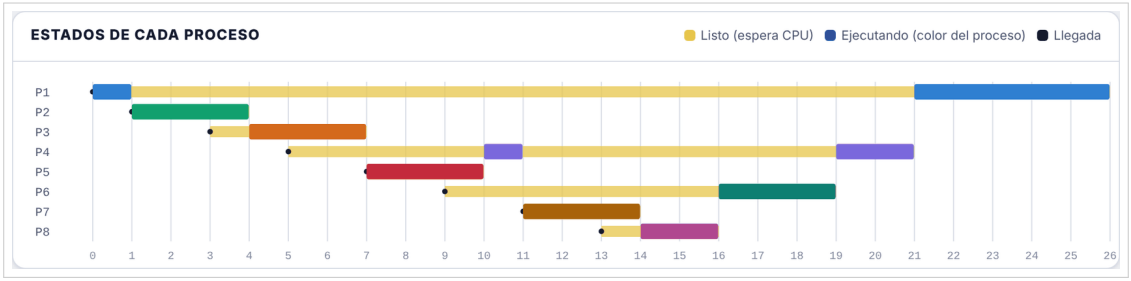
Inanición con prioridad expulsiva. En el ejemplo «Inanición por prioridad», P1 (prioridad 5) empieza en $t = 0$ pero cada proceso de prioridad 1 o 2 que llega lo expulsa; termina con 20 u de espera, frente a 0 u con Prioridad no expulsiva (Figura 78).

Figura 78
Inanición de P1 con Prioridad expulsiva

(a) Diagrama de Gantt



(b) Estados de cada proceso



Nota. P1 pasa casi toda la simulación en la cola de listos (franja amarilla).

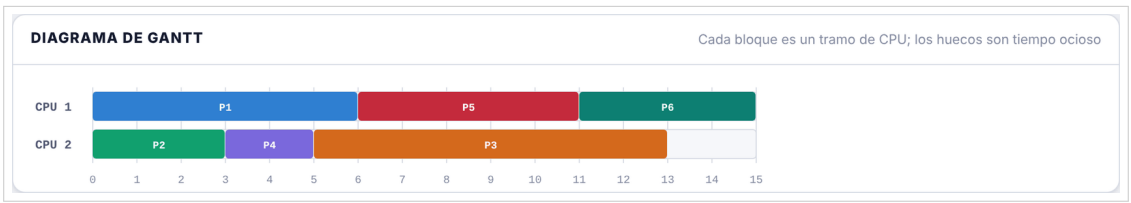
Varias CPU. Con 2 CPU y HRRN, dos procesos ejecutan a la vez y el Gantt tiene una fila por CPU (Figura 79).

Figura 79
Planificación con dos CPU y HRRN

(a) Estado en $t = 6$



(b) Diagrama de Gantt final



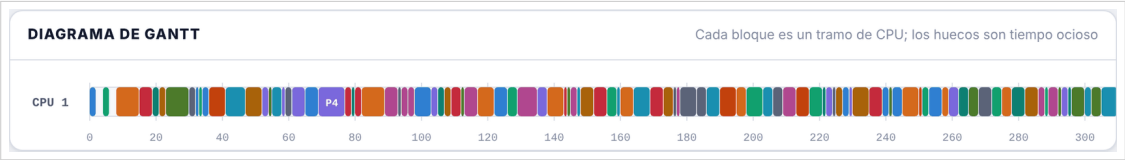
Nota. Cada CPU toma al proceso con mayor razón de respuesta cuando queda libre.

Mil procesos. El archivo `planificacion_1000.txt` carga 1 000 procesos. Con SRT terminan en 4 583 u con 802,82 u de espera promedio; el Gantt se desplaza horizontalmente y la comparación de las 7 políticas se calcula en menos de un segundo (Figura 80).

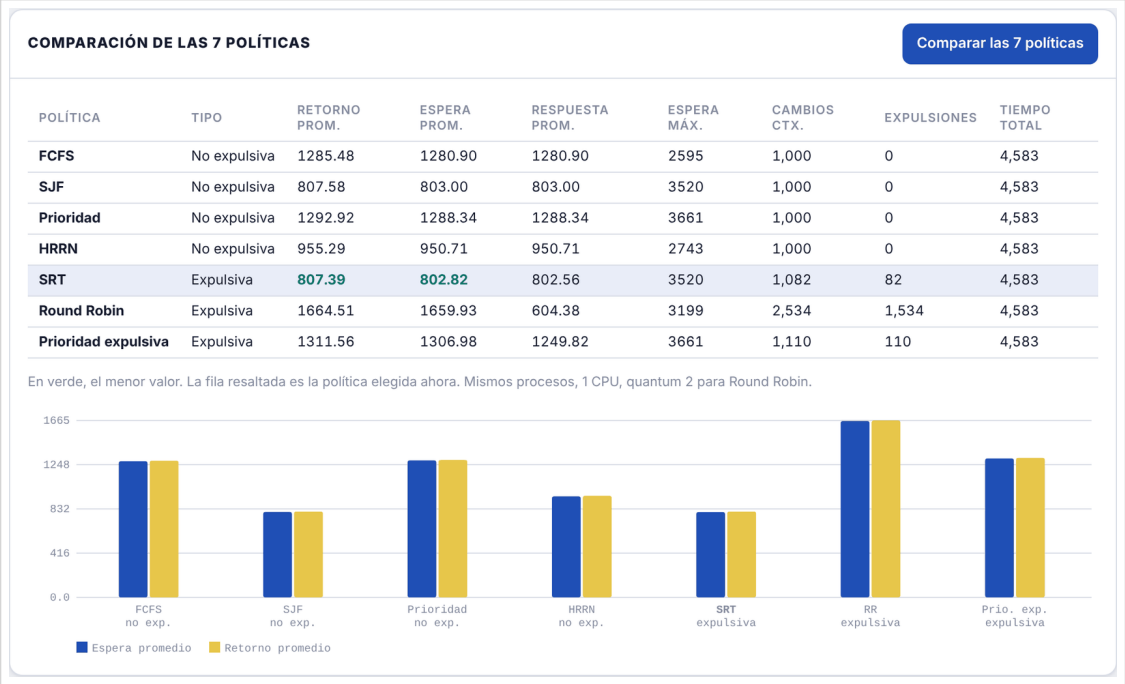
Figura 80

Planificación de 1 000 procesos cargados desde un archivo

(a) Diagrama de Gantt con SRT



(b) Comparación de las 7 políticas



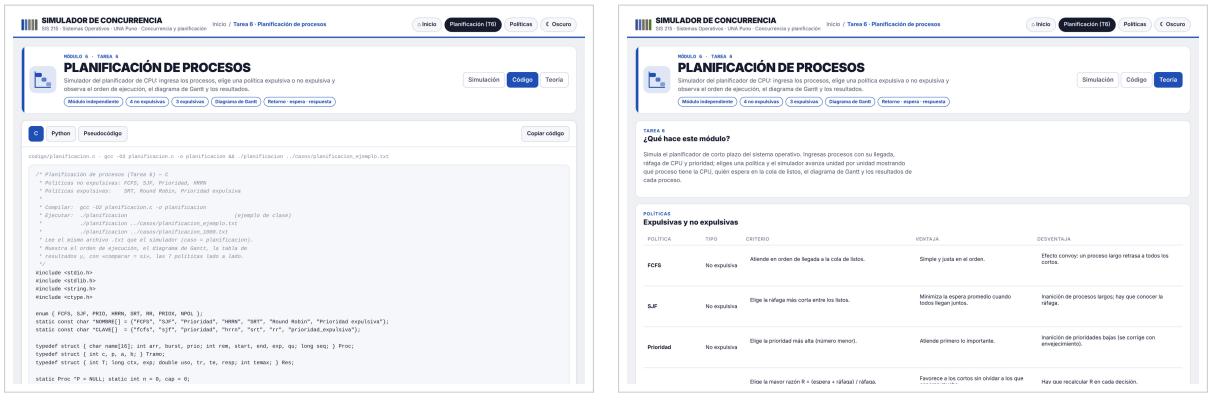
Nota. Con miles de procesos solo cambian la escala del Gantt y el número de filas mostradas.

8.8 Código, teoría y tema oscuro

Figura 81

Pestañas de código y teoría del módulo 6

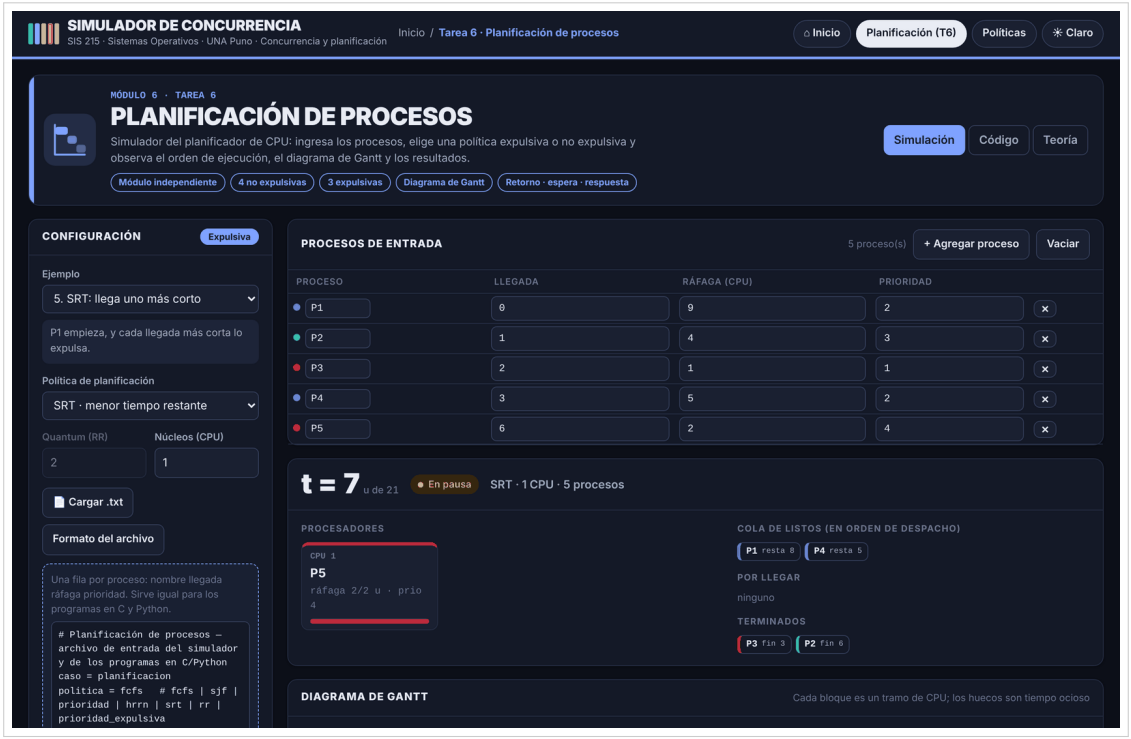
(a) Código *(b) Teoría*



Nota. El código en C y Python es el de `codigo/planificacion.c` y `codigo/planificacion.py`; la teoría explica las políticas, las fórmulas y las reglas de desempate.

Figura 82

Módulo 6 en tema oscuro con SRT



Nota. Todos los colores se adaptan al tema oscuro.

9. Resultados: comparación de las políticas de planificación

Para cada caso se ejecutó la misma configuración con las siete políticas usando el botón «Comparar las 7 políticas» (Figuras 17, 28, 39, 50 y 61). Se usaron 2 núcleos,

salvo en los fumadores, donde se usó 1 para que el agente y los fumadores compitan por la CPU. La Tabla 14 resume el tiempo total, los cambios de contexto, las expulsiones y la espera en la cola de listos.

Tabla 14

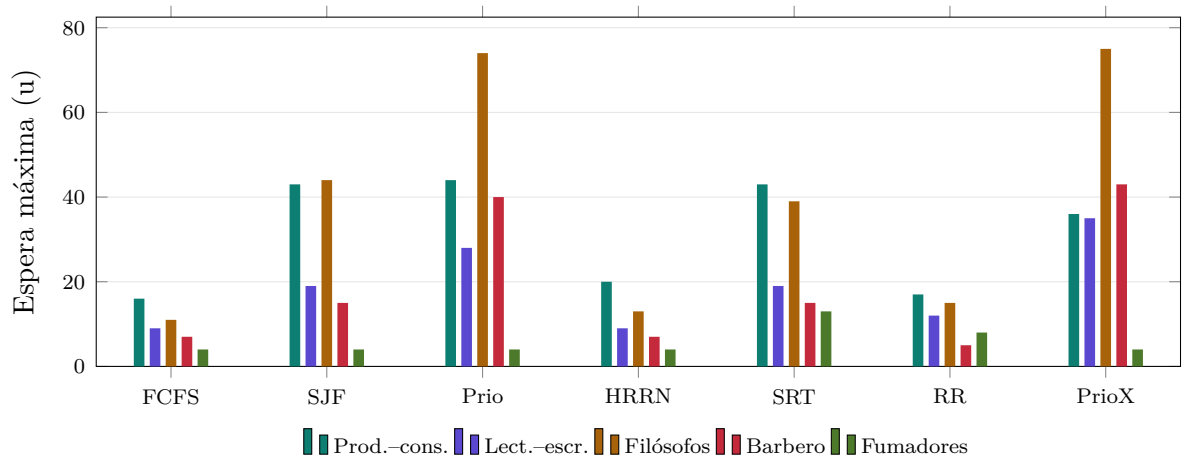
Comparación de las siete políticas de planificación de CPU en los cinco casos

Caso	Política	T	Cambios ctx.	Expulsiones	Espera prom.	Espera máx.	Uso CPU
Productor–consumidor	FCFS	88	43	0	9.5	16	98 %
	SJF	84	43	0	6.1	43	97 %
	Prioridad	89	44	0	5.1	44	95 %
	HRRN	86	43	0	9.1	20	98 %
	SRT ⋆	85	45	1	7.2	43	99 %
	Round Robin ⋆	89	95	53	9.1	17	98 %
	Prioridad expulsiva ⋆	78	49	5	4.1	36	99 %
Lectores–escritores	FCFS	88	61	0	3.6	9	89 %
	SJF	89	59	0	2.4	19	88 %
	Prioridad	91	57	0	2.9	28	85 %
	HRRN	93	57	0	1.8	9	87 %
	SRT ⋆	89	59	0	2.4	19	88 %
	Round Robin ⋆	97	81	27	2.9	12	79 %
	Prioridad expulsiva ⋆	88	60	3	3.1	35	90 %
Filósofos	FCFS	96	58	0	5.8	11	97 %
	SJF	100	58	0	5.6	44	94 %
	Prioridad	102	62	0	5.1	74	96 %
	HRRN	92	56	0	4.1	13	99 %
	SRT ⋆	105	74	13	4.9	39	91 %
	Round Robin ⋆	98	110	49	5.9	15	97 %
	Prioridad expulsiva ⋆	100	67	6	4.9	75	96 %
Barbero dormilón	FCFS	53	17	0	2.6	7	94 %
	SJF	55	18	0	2.4	15	92 %
	Prioridad	52	17	0	2.5	40	94 %
	HRRN	53	17	0	2.6	7	94 %
	SRT ⋆	55	18	0	2.4	15	92 %
	Round Robin ⋆	46	80	64	2.4	5	96 %
	Prioridad expulsiva ⋆	49	18	1	2.5	43	97 %
Fumadores (1 núcleo)	FCFS	115	40	0	1.3	4	100 %
	SJF	115	40	0	1.3	4	100 %
	Prioridad	115	40	0	1.3	4	100 %
	HRRN	115	40	0	1.3	4	100 %
	SRT ⋆	113	50	10	1.8	13	100 %
	Round Robin ⋆	108	61	21	2.1	8	100 %
	Prioridad expulsiva ⋆	115	40	0	1.3	4	100 %

Nota. ⋆ = política expulsiva. Las esperas se refieren a la cola de listos, en unidades de tiempo. Se usaron 2 núcleos (1 en los fumadores) y la misma semilla para las siete políticas.

Figura 83

Espera máxima en la cola de listos según la política de planificación



Nota. Datos de la Tabla 14. SJF, SRT y las políticas de prioridad postergan a las ráfagas largas o de prioridad baja.

Los resultados permiten cuatro observaciones:

1. **Las no expulsivas nunca expulsan.** FCFS, SJF, Prioridad y HRRN registran cero expulsiones en todos los casos, como exige su definición.
2. **Round Robin multiplica los cambios de contexto.** En todos los casos produce el mayor número de cambios de contexto y de expulsiones, pero mantiene baja la espera máxima: es la política más equitativa.
3. **SJF, SRT y Prioridad producen inanición.** En los cuatro primeros casos, su espera máxima en la cola de listos es varias veces la de FCFS o HRRN, porque las ráfagas largas o de prioridad baja son postergadas.
4. **HRRN equilibra.** Mantiene una espera máxima cercana a la de FCFS sin expulsar a nadie, porque la razón de respuesta crece con la espera.

Para el módulo 6, la Tabla 15 compara las siete políticas con los 1 000 procesos de `planificacion_1000.txt`. Como todos los procesos usan la misma CPU sin huecos, el tiempo total es igual (4 583 u) y solo cambia el reparto de la espera.

Tabla 15
Comparación de las siete políticas con 1 000 procesos

Política	Retorno prom.	Espera prom.	Respuesta prom.	Espera máx.	Cambios ctx.	Expuls.
FCFS	1285,48	1280,90	1280,90	2595	1000	0
SJF	807,58	803,00	803,00	3520	1000	0
Prioridad	1292,92	1288,34	1288,34	3661	1000	0
HRRN	955,29	950,71	950,71	2743	1000	0
SRT ⋆	**807,39**	**802,82**	802,56	3520	1082	82
Round Robin ⋆	1664,51	1659,93	**604,38**	3199	2534	1534
Prioridad expulsiva ⋆	1311,56	1306,98	1249,82	3661	1110	110

Nota. ⋆ = política expulsiva. 1 CPU, quantum 2. Tiempos en unidades; en negrita, el mejor valor. Resultados idénticos en el simulador, en C y en Python.

SRT y SJF minimizan la espera promedio, pero su espera máxima (3 520 u) es mayor que la de FCFS (2 595 u): los procesos largos quedan relegados. HRRN queda entre ambos extremos. Round Robin da la mejor respuesta, a cambio de 2 534 cambios de contexto.

La planificación también interactúa con la sincronización. En lectores–escritores con prioridad expulsiva, un lector de prioridad baja puede perder la CPU mientras retiene el recurso, y un escritor de prioridad alta queda esperando: es una forma de *inversión de prioridad* (Silberschatz et al., 2018). En los filósofos, un filósofo que come sin CPU retiene sus tenedores más tiempo y retrasa a sus vecinos.

10. Programas en C y Python

La carpeta `codigo` del repositorio contiene los cinco problemas en C con `pthreads` y en Python con `threading`. Todos leen el mismo archivo de entrada que el simulador, usan memoria dinámica y pilas de hilo pequeñas para admitir miles de hilos, y en las versiones ingenuas incluyen un hilo vigilante que detecta el interbloqueo y termina el programa con código 2. El planificador de CPU se implementa con la función `cpu_trabajar` (Listado 4): con políticas no expulsivas un hilo recibe un núcleo para toda su ráfaga; con las expulsivas lo recibe por tramos (una unidad en SRT y Prioridad expulsiva, un quantum en Round Robin), y después de cada tramo se vuelve a elegir al mejor hilo listo.

Listado 4
Núcleo del planificador de CPU en C (fragmento)

```
1 static void cpu_trabajar(int id, int u) {
2     if (NUCLEOS <= 0) { dormir_u(u); return; }          /* sin límite de nú
          cleos */
3     Tarea *t = &tareas[id];
```

```
    pthread_mutex_lock(&mcpu);
    t->rafaga = u; t->resta = u; t->desde = ahora_u(); t->listo = 1;
    despachar();                                          /* elige según la
        política */
    while (t->resta > 0) {
        while (!t->corriendo) pthread_cond_wait(&t->cv, &mcpu);
        int tramo = PLAN == P_RR ? min(QUANTUM, t->resta)
                  : (PLAN == P_SRT || PLAN == P_PRIOX) ? 1 : t->resta;
        pthread_mutex_unlock(&mcpu);
        dormir_u(tramo);                                  /* ejecuta en su nú
            cleo */
        pthread_mutex_lock(&mcpu);
        t->resta -= tramo; t->corriendo = 0; libres++;
        if (t->resta > 0) { t->listo = 1; t->desde = ahora_u(); }
        despachar();                                      /* puede expulsarlo
             */
    }
    pthread_mutex_unlock(&mcpu);
}
```

Para la Tarea 6, `codigo/planificacion.c` y `codigo/planificacion.py` implementan el mismo algoritmo del módulo 6: leen la lista de procesos, simulan una unidad de tiempo por vuelta e imprimen el orden de ejecución, un diagrama de Gantt en texto, la tabla de resultados y la comparación de las siete políticas (Figura 85).

La Tabla 16 resume la ejecución de los programas con los archivos de ejemplo del repositorio en Linux con GCC y Python 3. La Figura 84 muestra la salida del barbero con atención SRT y planificador HRRN.

Tabla 16

Ejecución de los programas en C y Python con los archivos de ejemplo

Archivo de entrada	C (s)	Python (s)	Código	Resultado (programa en C)
barbero_1000.txt	1.0	1.1	0	✓ La sala nunca superó sus 25 sillas (máximo observado: 25).
filosofos_1000.txt	0.1	0.4	0	✓ Todos comieron 3 veces. Sin interbloqueo. Tiempo total: 90.2 u
fumadores_1000.txt	2.0	2.1	0	✓ 1000 rondas sin interbloqueo. Tiempo total: 1962.4 u
lectores_escritores_1000.txt	2.7	2.1	0	✓ Sin lecturas sucias ni escrituras perdidas.
productor_consumidor_1000.txt	0.1	0.4	0	✓ Cada ítem se consumió exactamente una vez.
barbero_lista_2000.txt	2.0	2.1	0	✓ La sala nunca superó sus 30 sillas (máximo observado: 17).
fumadores_lista_1500.txt	2.8	3.0	0	✓ 1500 rondas sin interbloqueo. Tiempo total: 2777.9 u
barbero_planificador.txt	0.4	0.5	0	✓ La sala nunca superó sus 15 sillas (máximo observado: 15).
filosofos_planificador.txt	0.2	0.3	0	✓ Todos comieron 4 veces. Sin interbloqueo. Tiempo total: 205.8 u
fumadores_planificador.txt	1.2	1.3	0	✓ 200 rondas sin interbloqueo. Tiempo total: 1205.9 u
lectores_escritores_planificador.txt	0.4	0.4	0	✓ Sin lecturas sucias ni escrituras perdidas.
productor_consumidor_planificador.txt	0.4	0.4	0	✓ Cada ítem se consumió exactamente una vez.
barbero_sin_mutex.txt	0.0	0.1	0	△ Condición de carrera: llegó a haber 8 clientes en una sala de 3 sillas.
productor_consumidor_sin_mutex.txt	0.0	0.2	0	△ Condición de carrera: el búfer quedó inconsistente.
lectores_escritores_sin_sincronizar.txt	0.0	0.1	0	△ Condición de carrera: datos inconsistentes.
filosofos_ingenua.txt	2.4	2.4	2	× INTERBLOQUEO detectado: nadie come desde hace 2200 ms. Cada filósofo retiene un tenedor y espera el del vecino.
fumadores_ingenua.txt	2.4	2.4	2	× INTERBLOQUEO detectado: los ingredientes quedaron repartidos entre varios fumadores; nadie completa su cigarro y el agente espera para siempre.
planificacion_ejemplo.txt	0.0	0.0	0	✓ Retorno prom: 17.00 u | Espera prom: 11.40 u
planificacion_convoy.txt	0.0	0.0	0	✓ Retorno prom: 22.20 u | Espera prom: 16.60 u
planificacion_round_robin.txt	0.0	0.0	0	✓ Retorno prom: 14.50 u | Espera prom: 9.75 u
planificacion_srt.txt	0.0	0.0	0	✓ Retorno prom: 7.80 u | Espera prom: 3.60 u
planificacion_prioridad_expulsiva.txt	0.0	0.0	0	✓ Retorno prom: 8.50 u | Espera prom: 5.25 u
planificacion_2cpu.txt	0.0	0.0	0	✓ Retorno prom: 7.17 u | Espera prom: 2.50 u
planificacion_1000.txt	0.0	0.9	0	✓ Retorno prom: 807.39 u | Espera prom: 802.82 u

Nota. Tiempo real de ejecución con `escala_ms = 1`. Código de salida 2: el vigilante detectó el interbloqueo. ✓ = correcto; △ = condición de carrera esperada; × = interbloqueo esperado.

Figura 84

Salida del programa en C del barbero con atención SRT y 2 núcleos HRRN

```
$ ./barbero ../casos/barbero_planificador.txt
Barbero dormilón: 4 barberos, 15 sillas, 300 clientes · política SRT (expulsiva)
Planificador de CPU: 2 núcleo(s) · HRRN

Atendidos: 105 | Rechazados: 195 | Espera prom: 55.3 u | Espera máx: 362.6 u | Expulsiones de clientes:
50 | Uso barberos: 43.3 %
CPU · HRRN: despachos 713 | expulsiones 0 | espera en listos prom 1.1 u, máx 2.1 u | uso de CPU 86.5 %
✓ La sala nunca superó sus 15 sillas (máximo observado: 15).
```

Nota. Por la naturaleza concurrente, los valores cambian ligeramente entre ejecuciones.

Figura 85

Salida del programa en C de planificación de procesos con Round Robin (quantum 2)

```
$ ./planificacion ../casos/planificacion_round_robin.txt
Planificación de procesos: 4 procesos · Round Robin (expulsiva) · quantum 2 · 1 CPU

Orden de ejecución:
A[0-2] → B[2-4] → C[4-6] → D[6-8] → A[8-10] → B[10-11] → C[11-13] → D[13-15] → A[15-16] →
C[16-19]

Diagrama de Gantt (1 carácter = 1 u)
CPU1 |1122334411233441333|
0                 19
1=A  2=B  3=C  4=D

Proceso   Lleg  Ráf  Prio  Inicio   Fin  Retorno  Espera  Resp  Exp
A            0     5     2       0    16       16      11     0    2
B            0     3     1       2    11       11       8     2    1
C            1     7     3       4    19       18      11     3    2
D            2     4     2       6    15       13       9     4    1

Retorno prom: 14.50 u | Espera prom: 9.75 u | Respuesta prom: 2.25 u | Espera máx: 11 u
Tiempo total: 19 u | Cambios de contexto: 10 | Expulsiones: 6 | Uso de CPU: 100.0 %

Política                Retorno   Espera   Resp.  Esp.máx   Ctx   Exp      T
FCFS                      11.00     6.25    6.25       13     4     0     19
SJF                        9.50     4.75    4.75       11     4     0     19
Prioridad                  9.75     5.00    5.00       11     4     0     19
HRRN                      10.25     5.50    5.50       11     4     0     19
SRT                        9.50     4.75    4.75       11     4     0     19
Round Robin               14.50     9.75    2.25       11    10     6     19
Prioridad expulsiva        9.75     5.00    5.00       11     4     0     19
✓ Todos los procesos terminaron.
```

Nota. La salida es determinista: el simulador, el programa en C y el programa en Python dan los mismos valores.

11. Políticas de uso y privacidad

El simulador incluye políticas de uso y privacidad que deben aceptarse en la pantalla de inicio y pueden consultarse en todo momento (Figura 86).

11.1 Políticas de uso

1. **Finalidad.** Herramienta educativa del curso SIS 215 Sistemas Operativos para mostrar cómo hilos, semáforos, mutex y monitores resuelven cinco problemas clásicos de concurrencia, y cómo el planificador ordena los procesos según su política (módulo 6).
2. **Uso permitido.** Puede usarse, exponerse y modificarse con fines académicos, citando la fuente.
3. **Alcance.** Las simulaciones avanzan por unidades de tiempo discretas; ilustran el comportamiento de los algoritmos y no miden el rendimiento de un sistema operativo real.
4. **Archivos de entrada.** Los archivos .txt se leen en el navegador y no se envían a ningún servidor; deben contener solo datos de prueba.
5. **Datos ficticios.** Todos los hilos y procesos son generados por el programa a partir de una semilla o del archivo cargado.

11.2 Política de privacidad

1. **Sin datos personales.** El simulador no pide nombre, correo ni ningún otro dato del usuario.
2. **Procesamiento local.** Las simulaciones y la lectura de archivos se hacen en el navegador; nada se envía a un servidor.
3. **Almacenamiento.** Solo se guarda la preferencia de tema en el almacenamiento local del navegador.
4. **Terceros.** Las tipografías se descargan de Google Fonts, que puede registrar la dirección IP según su propia política.
5. **Marco legal.** Si una versión futura registrara datos de personas reales, se aplicaría la Ley N.º 29733 (Congreso de la República del Perú, 2011) y su reglamento (Ministerio de Justicia y Derechos Humanos, 2024): consentimiento previo, finalidad declarada, derechos ARCO y medidas de seguridad.

Figura 86
Políticas de uso y privacidad dentro del simulador



Nota. Se aceptan en la pantalla de inicio antes de entrar a cualquier caso.

12. Conclusiones

1. Los cinco problemas clásicos se resuelven con combinaciones de semáforos, mutex y monitores, y las mismas soluciones funcionan para cualquier número de hilos: el simulador y los programas se probaron con miles de hilos sin pérdida de datos ni bloqueos.

2. La exclusión mutua es indispensable. Sin ella aparecieron ítems perdidos en el productor–consumidor, lecturas sucias y escrituras perdidas en lectores–escritores, y clientes sin silla en el barbero.

3. El interbloqueo surge cuando se cumplen las cuatro condiciones de Coffman. Las soluciones ingenuas de los filósofos y de los fumadores lo alcanzaron; romper una sola condición (orden de recursos, límite de sillas, adquisición atómica o empujadores) lo eliminó.

4. La inanición depende de la política: la prioridad a lectores o a escritores, SJF, SRT y la prioridad estática postergan a ciertos hilos, mientras que el torniquete, HRRN, Round Robin y el envejecimiento la reducen.

5. Las políticas expulsivas mejoran la respuesta de los trabajos cortos o urgentes,

pero aumentan los cambios de contexto; Round Robin fue la más equitativa y SRT la de menor espera promedio en la atención del barbero.

6. La planificación interactúa con la sincronización: un hilo que pierde la CPU mientras retiene un recurso retrasa a todos los que lo esperan, como en la inversión de prioridad.

7. El módulo de planificación de procesos cumple los requisitos de la Tarea 6: ingreso de procesos, políticas expulsivas y no expulsivas, orden de ejecución, cálculo de retorno, espera y respuesta, diagrama de Gantt e interfaz gráfica. Con el ejemplo de clase, SRT dio la menor espera promedio (6,60 u) y Round Robin la menor respuesta promedio (2,80 u), pero la mayor espera (13,00 u).

8. Ninguna política es la mejor en todo: las que minimizan la espera promedio (SJF, SRT) castigan a los procesos largos, y las más equitativas (Round Robin) aumentan los cambios de contexto. La elección depende de qué se quiere optimizar.

9. Reunir los cinco casos en módulos independientes, con archivos de entrada comunes al simulador y a los programas, permite comparar los fenómenos y reproducir cada experimento.

Referencias

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). *Operating systems: Three easy pieces* (versión 1.10). Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/

Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System deadlocks. *ACM Computing Surveys, 3*(2), 67–78. https://doi.org/10.1145/356586.356588

Congreso de la República del Perú. (2011, 3 de julio). *Ley N.º 29733, Ley de Protección de Datos Personales*. Diario Oficial El Peruano. https://www.congreso.gob.pe/Docs/DGP/DIDP/files/ley_29733.pdf

Courtois, P. J., Heymans, F., & Parnas, D. L. (1971). Concurrent control with "readers" and "writers". *Communications of the ACM, 14*(10), 667–668. https://doi.org/10.1145/362759.362813

Dijkstra, E. W. (1965). *Cooperating sequential processes* (EWD 123) [Manuscrito]. E. W. Dijkstra Archive, The University of Texas at Austin. https://www.cs.utexas.edu/~EWD/transcriptions/EWD01xx/EWD123.html

Downey, A. B. (2016). *The little book of semaphores* (2.ª ed., versión 2.2.1). Green Tea

Press. https://greenteapress.com/wp/semaphores/

Hoare, C. A. R. (1974). Monitors: An operating system structuring concept. *Communications of the ACM, 17*(10), 549–557. https://doi.org/10.1145/355620.361161

Mandl, P. (2020). *Grundkurs Betriebssysteme: Architekturen, Betriebsmittelverwaltung, Synchronisation, Prozesskommunikation, Virtualisierung* (5.ª ed.). Springer Vieweg. https://doi.org/10.1007/978-3-658-30547-5

Maziero, C. A. (2019). *Sistemas operacionais: Conceitos e mecanismos*. Editora da UFPR. https://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start

Ministerio de Justicia y Derechos Humanos. (2024). *Decreto Supremo N.º 016-2024-JUS, que aprueba el Reglamento de la Ley N.º 29733, Ley de Protección de Datos Personales*. Plataforma digital única del Estado peruano. https://www.gob.pe/institucion/anpd/normas-legales/6554453-16-2024-jus

Parnas, D. L. (1975). On a solution to the cigarette smoker's problem (without conditional statements). *Communications of the ACM, 18*(3), 181–183. https://doi.org/10.1145/360680.360709

Ruelas, D. (2026). *Sesión 05: Threads* [Diapositivas de clase]. Curso SIS 215 Sistemas Operativos, Universidad Nacional del Altiplano.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4.ª ed.). Pearson.

Wolf, G., Ruiz, E., Bergero, F., & Meza, E. (2015). *Fundamentos de sistemas operativos*. Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas y Facultad de Ingeniería. https://sistop.org/pdf/sistemas_operativos.pdf

Anexo A. Repositorio, video y ejecución

El simulador, los programas y los archivos de ejemplo están en:

https://github.com/Enerdev/Sisemas-Operativos

El producto entregable es el archivo `index.html`: se abre con doble clic en cualquier navegador moderno y no necesita instalación ni conexión a internet. El video con la explicación y la demostración del simulador está en:

https://drive.google.com/drive/folders/1wSo-oogrrC6L8_PlAP1vdnMhg8DpWUAO?usp=sharing

Tabla 17
Contenido del repositorio

Ruta	Contenido
`index.html`	Simulador completo en un solo archivo; se abre con cualquier navegador.
`casos/`	30 archivos .txt de ejemplo: los cinco casos (pequeños, de 1 000 o más hilos, con listas y con planificador) y 7 de planificación de procesos.
`codigo/`	Los cinco problemas y el planificador de procesos en C (`.c`) y en Python (`.py`).
`src/`	Código fuente del simulador: núcleo, planificador, ilustraciones, un archivo por caso y `plan.js` (módulo 6).
`tools/`	Partes comunes de los programas en C y Python.
`README.md`	Instrucciones de uso.

Listado 5
Compilación y ejecución de los programas

```
cd codigo
gcc -O2 barbero.c -o barbero -pthread
./barbero ../casos/barbero_1000.txt            # C con pthreads
python3 barbero.py ../casos/barbero_lista_2000.txt   # Python con threading
gcc -O2 planificacion.c -o planificacion
./planificacion ../casos/planificacion_ejemplo.txt    # Tarea 6
```

Anexo B. Claves de los archivos de entrada

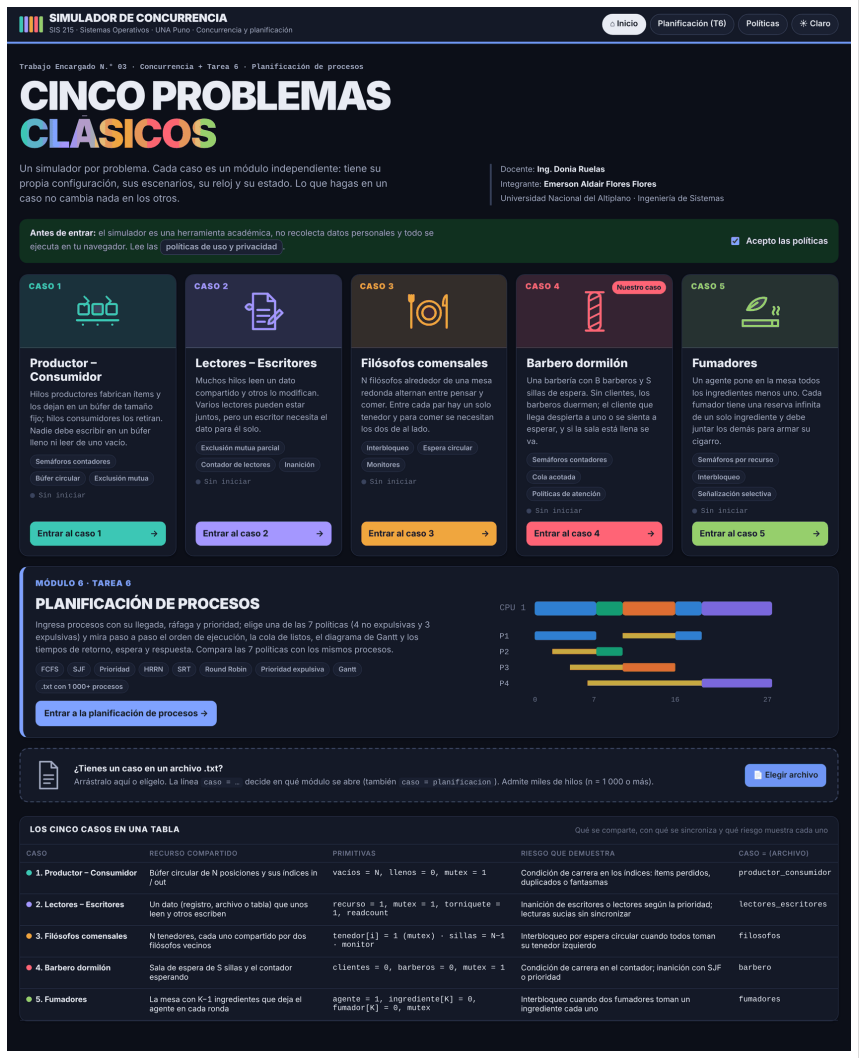
Tabla 18
Claves admitidas por cada caso

caso =	Claves
`productor_consumidor`	productores, consumidores, buffer, items, producir, consumir, mutex
`lectores_escritores`	lectores, escritores, accesos, pensar, leer, escribir, politica (lectores, escritores, justa, ninguna)
`filosofos`	filosofos, comidas, pensar, comer, estrategia (ingenua, asimetrica, camarero, monitor)
`barbero`	barberos, sillas, clientes, llegada, corte, politica, quantum_atencion, envejecimiento, mutex, `lista:` opcional
`fumadores`	ingredientes (3 a 6), rondas, agente, fumar, solucion (ingenua, dirigida, empujadores), `lista:` opcional
`planificacion`	politica, quantum, nucleos (1 a 8), `lista:` con nombre llegada ráfaga prioridad, o procesos/llegada/rafaga/prioridad/semilla para generarlos
`(todos)`	nucleos, planificador (fcfs, sjf, prioridad, hrrn, srt, rr, prioridad_expulsiva), quantum, semilla, escala_ms, detalle

Nota. Los rangos se escriben con dos números (mínimo y máximo); los valores sí/no aceptan «si» y «no».

Anexo C. Tema oscuro de la pantalla de inicio

Figura 87
Pantalla de inicio en tema oscuro



Nota. El tema sigue la preferencia del sistema y puede cambiarse con el botón superior.